

ESCUELA DE
INGENIERÍA INFORMÁTICA
PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

EVALUACIÓN MODULAR 3

LUCAS AHUMADA R.
NICOLÁS JARAMILLO O.
ETHAN TRONCOSO N.

INF2223-2
Junio 2026

ÍNDICE GENERAL

1. INTRODUCCIÓN	1
2. DOMINIO DEL PROBLEMA	2
3. ANÁLISIS	3
3.1 Requerimientos Funcionales	3
3.2 Requerimientos No Funcionales	3
3.3 Restricciones	4
3.4 Estructuras de datos utilizadas	4
4. DISEÑO	6
4.1. Arquitectura del Sistema	6
4.2. Arquitectura de Módulos	6
4.3. Herramientas y Tecnologías	7
4.4. Diagramas y Modelos Explicativos	8
5. PLANIFICACIÓN	11
5.1. Planificación grupal	11
5.2. Planificación individual	12
6. CONCLUSIÓN	13
ANEXO	14

1. INTRODUCCIÓN

La finalidad del presente informe es describir el desarrollo del sistema de gestión realizado para el parque de diversiones IBCLandia, el cual ha sido implementado en lenguaje C como parte de la Evaluación Modular 3 de Estructura de Datos. El proyecto busca representar el funcionamiento básico de un parque temático de manera digital, por lo que considera elementos como zonas, atracciones, visitantes, entradas y filas de espera, a su vez permitiendo administrar todos los datos mencionados mediante un menú diseñado para funcionar en la terminal.

Para el desarrollo del sistema se utilizaron diferentes estructuras de datos vistas durante la dictación del curso. El parque fue organizado utilizando un arreglo de zonas, donde cada zona contiene una lista doblemente enlazada de atracciones. A la vez, cada atracción contiene una fila de espera implementada mediante una lista simplemente enlazada. Las entradas del parque se almacenan en un árbol binario de búsqueda, el cual permite que sean organizadas y buscadas por medio de su ID. También se ha utilizado una lista de usuarios para poder registrar a los visitantes del parque y para que sean asociados con sus entradas y las filas a las que pertenecen.

El programa permite realizar todas las operaciones requeridas para la entrega, las cuales son: agregar, buscar, modificar, eliminar y listar todos sus datos. Entre las funcionalidades se encuentran la creación de zonas y atracciones, la administración de los visitantes, la gestión de las entradas, el ingreso de usuarios a las filas de las atracciones, la búsqueda de un usuario específico dentro de las filas y la simulación de vueltas de una atracción. Esta última funcionalidad permite representar el ciclo de uso de una atracción, llevando a los usuarios desde la fila hacia la ocupación actual de la atracción y liberando dicha ocupación cuando la vuelta termina.

A través de este proyecto desarrollado en 2 semanas de trabajo se busca aplicar de manera práctica todos los contenidos vistos en clase ya sea de estructuras estáticas, dinámicas simples y dinámicas complejas, respetando la lógica observada en los PDF disponibles en el aula virtual.

La prioridad al desarrollar este programa fue que el usuario pueda visualizar los datos de una manera intuitiva para facilitar su uso y comprensión durante la ejecución del mismo.

2. DOMINIO DEL PROBLEMA

El problema solicitado a resolver es la administración de la información principal del parque de IBCLandia. El parque necesita gestionar de manera ordenada sus zonas, atracciones, visitantes, entradas y filas de espera, ya que todos estos elementos se relacionan entre sí en el funcionamiento normal del parque y administrarlo de manera análoga sería muy complicado.

Sin un sistema organizado que utilice estructuras de datos estáticas y dinámicas como el desarrollado el manejo de estos datos podría volverse abrumador, especialmente al realizar asociaciones como registrar un visitante a una entrada, revisar el estado de una atracción o controlar una fila de espera. Por ejemplo, una atracción puede encontrarse en distintos estados los cuales influyen directamente en si los visitantes pueden o no ingresar a su fila. Del mismo modo, una entrada puede estar activa, utilizada, anulada o vencida, por lo que es necesario controlar su estado en tiempo real para evitar usos incorrectos de la misma.

Además, el parque requiere llevar un control de la ocupación de sus atracciones y zonas, lo cual conlleva una gran organización y precisión para representar correctamente a los visitantes dentro del recinto.

Abordar este problema es relevante, ya que permite aplicar los contenidos de Estructura de Datos en un caso práctico y cercano a una situación real. Este proyecto no solo permite organizar información, realizar búsquedas, modificaciones y eliminaciones, sino que también demuestra cómo las estructuras de datos pueden utilizarse para representar y resolver problemas de gestión en un entorno.

3. ANÁLISIS

En esta sección se detallan los requerimientos y el análisis del sistema de IBCLandia desarrollado para gestionar el parque de diversiones. El objetivo principal del software es poder gestionar a los usuarios, entradas, zonas, atracciones y filas del parque mediante el uso de estructura de datos implementado en ANSI C.

La captura de requerimientos se realizó considerando las principales necesidades que se necesitan para administrar un parque de diversiones, tales como registrar los visitantes, controlar las entradas, administrar las zonas y atracciones, manejar filas y poder generar reportes diarios sobre el funcionamiento del parque.

3.1 Requerimientos Funcionales

Los requerimientos funcionales son aquellas funcionalidades que debe permitir realizar el sistema al usuario. Para IBCLandia se consideraron los siguientes requerimientos:

1. El sistema debe permitir registrar, buscar, modificar, eliminar y listar usuarios.
2. El sistema debe permitir crear, modificar, buscar, eliminar y listar las entradas del parque.
3. El sistema debe permitir asociar usuarios a entradas y controlar su ingreso o salida del parque.
4. El sistema debe permitir agregar, buscar, modificar, eliminar y listar zonas del parque.
5. El sistema debe permitir agregar, buscar, modificar, eliminar y listar atracciones asociadas a una zona.
6. El sistema debe permitir gestionar filas de usuarios en las atracciones.
7. El sistema debe permitir iniciar y terminar vueltas de atracciones, actualizando su ocupación.
8. El sistema debe permitir generar reportes de ingresos, entradas vendidas, entradas usadas, visitantes, ocupación de zonas y atracciones no operativas.

3.2 Requerimientos No Funcionales

Los requerimientos no funcionales corresponden a las características de calidad que debe cumplir el sistema. Para IBCLandia, se consideran los siguientes:

1. El sistema debe ejecutarse mediante consola.
2. El sistema debe presentar menús claros y fáciles de usar.
3. El sistema debe validar datos importantes ingresados por el usuario como edad, estatura, horarios, capacidades, tipos y estados.
4. El sistema debe organizarse de forma modularizada es decir mediante funciones separadas para facilitar su comprensión y mantenimiento.
5. El sistema debe utilizar correctamente memoria dinámica, punteros y estructuras de datos.
6. El sistema debe manejar errores básicos, como datos inválidos, elementos inexistentes o punteros nulos.

7. El sistema debe responder adecuadamente en las operaciones principales de registro, búsqueda, modificación, eliminación y listado.

3.3 Restricciones

Las restricciones corresponden a las limitaciones técnicas y reglas bajo las cuales se desarrolló el sistema de IBCLandia, para este se consideraron las siguientes restricciones:

1. El sistema debe ser desarrollado en lenguaje ANSI C.
2. El sistema funciona y se utiliza sólo mediante consola, sin interfaz gráfica.
3. La información se mantiene en memoria durante la ejecución, sin base de datos ni almacenamiento permanente.
4. El parque tiene un máximo de 15 zonas.
5. Las atracciones deben estar asociadas a una zona existente.
6. Las filas deben estar asociadas a una atracción existente.
7. Un usuario debe estar dentro del parque para poder ingresar a una fila de atracción.
8. Los usuarios deben estar registrados para ser asociados a entradas o filas.
9. Una atracción debe estar operativa para permitir el ingreso de usuarios a su fila.
10. Un usuario no debe permanecer en más de una fila al mismo tiempo.
11. Las entradas poseen tipos definidos: normal, pase rápido, familiar e infantil.
12. Las entradas poseen estados definidos: activa, utilizada, anulada o vencida.
13. Los nombres de usuarios deben cumplir con el formato solicitado por el sistema.
14. Solo se permite ingresar usuarios a filas de atracciones operativas.
15. La edad y estatura de los usuarios deben encontrarse dentro de los rangos definidos por las validaciones del programa.
16. Se deben utilizar estructuras de datos como arreglos, listas enlazadas, listas doblemente enlazadas y árboles binarios de búsqueda.

3.4 Estructuras de Datos Utilizadas

El sistema almacena la información en memoria utilizando estructuras de datos en C. Cada estructura fue diseñada para representar un elemento importante del parque y sus relaciones.

Parque: La estructura Parque es la estructura principal de nuestro sistema. Desde ella se accede a las zonas, usuarios y entradas.

Contiene un arreglo de zonas, la cantidad de zonas registradas, el número de visitantes del día, una lista de usuarios y un árbol binario de búsqueda de entradas. Esta estructura funciona como el centro del sistema, ya que conecta los distintos módulos.

Arreglo estático de Zonas: Las zonas se almacenan en un arreglo de tamaño fijo. Cada posición del arreglo contiene un puntero a una zona creada dinámicamente.

Esto permite acceder directamente a las zonas mediante posiciones del arreglo y controlar la cantidad máxima de zonas permitidas. En este caso, el sistema define un máximo de 15 zonas.

Cada zona contiene sus propios datos y una lista de atracciones asociadas.

Lista Doblemente Enlazada de Atracciones: Cada zona posee una lista doblemente enlazada de atracciones que emplea nodo fantasma. Esta lista permite recorrer fácilmente las atracciones de una zona y también eliminar nodos actualizando los enlaces anterior y siguiente.

Cada nodo de la lista contiene un puntero a una atracción. Esta estructura es útil porque una zona puede tener distintas cantidades de atracciones, por lo que no se requiere un arreglo fijo para almacenarlas.

Cada atracción tiene una lista de espera asociada.

Lista Simplemente Enlazada de Fila: Cada atracción posee una fila de usuarios, implementada mediante una lista simplemente enlazada la cual cada Nodo apunta mediante un puntero hacia la estructura del usuario perteneciente a ese espacio en la fila. Esta estructura representa el orden de llegada de los usuarios que esperan para ingresar a una atracción.

Cuando se inicia una vuelta en la atracción, los usuarios se retiran desde el inicio de la fila y pasan a ocupar la atracción hasta completar su capacidad máxima. Esta estructura permite manejar filas de tamaño variable según la cantidad de usuarios que esperen.

Lista de usuarios: Los usuarios se almacenan en una lista simplemente enlazada con uso de nodo fantasma. Esta estructura permite agregar usuarios de forma dinámica sin definir previamente una cantidad máxima.

Cada nodo de la lista apunta a un usuario. Gracias a esta lista, el sistema puede recorrer todos los usuarios registrados, buscar un usuario por id, modificarlo, eliminarlo o verificar si está dentro del parque.

ABB de entradas: Las entradas se almacenan en un árbol binario de búsqueda con nodo fantasma. Cada nodo del árbol contiene una entrada y se ordena según el id de la entrada.

Esta estructura permite organizar las entradas de forma jerárquica y realizar búsquedas, inserciones, eliminaciones y listados mediante recorridos del árbol. Además, los reportes de entradas vendidas, entradas usadas e ingresos totales se calculan recorriendo este ABB.

4. DISEÑO

La solución propuesta consiste en un sistema de gestión por consola, implementado en lenguaje C, el cual modela las distintas entidades del parque temático IBCLandia mediante estructuras de datos dinámicas y estáticas. La solución es un sistema completamente interactivo mediante un menú, con funcionalidades de creación, modificación, lectura y eliminación sobre los elementos esenciales del parque temático.

4.1. Arquitectura del Sistema

El sistema se organiza en torno a la estructura central del Parque, que contiene las demás entidades. Las zonas son almacenadas en un arreglo estático de hasta 15 punteros (zonas[MAX_ZONAS]), lo que permite acceso directo por índice y compactación al eliminar. Cada zona contiene una lista doblemente enlazada de atracciones con nodo fantasma. Cada atracción contiene una lista simplemente enlazada que representa la lista de espera de dicha atracción. Los usuarios del parque se almacenan en una lista simplemente enlazada con nodo fantasma a nivel global del Parque. Las entradas se organizan en un árbol binario de búsqueda por ID, lo que permite búsquedas computacionalmente eficientes. Cada entrada tiene su propia lista de usuarios vinculados a dicha entrada por medio de punteros que los referencian, la memoria de estas estructuras de datos se gestiona dinámicamente mediante el uso de malloc, según la práctica estándar del curso.

4.2. Arquitectura de Módulos

El sistema se organiza en módulos funcionales, cada uno asociado a un menú principal. Esta separación permite ordenar el código y facilitar la mantención del programa.

Los módulos del sistema son:

1. **Menú principal:** El menú principal funciona como punto de entrada del sistema. Desde este menú, el usuario puede acceder a las distintas áreas del programa, como usuarios, entradas, zonas, atracciones, filas y reportes, este menú permite centralizar la navegación del sistema y evita que el usuario deba acceder directamente a las funciones internas del código.
2. **Menú de usuarios:** El módulo de usuarios se encarga de administrar la información de los visitantes del parque. Permite registrar nuevos usuarios, buscarlos por identificador, modificar sus datos, eliminarlos y listarlos, cada usuario posee datos como nombre, id, edad, estatura, si se encuentra actualmente dentro del parque y si estuvo previamente en el parque. Este módulo también participa en el control de ingreso y salida de visitantes.
3. **Menú de entradas:** El módulo de entradas permite administrar las entradas vendidas por el parque. Cada entrada posee un id, valor, tipo y estado, este módulo permite agregar nuevas entradas, buscar entradas existentes, modificar sus datos, cambiar su estado,

eliminarlas y listar la lista. Además, permite asociar usuarios a una entrada, lo cual es útil para representar entradas individuales o familiares.

4. **Menú de zonas:** El módulo de zonas administra las distintas áreas del parque. Cada zona contiene información como nombre, código, temática, capacidad máxima, ocupación actual y horario, desde este módulo se pueden agregar, buscar, modificar, eliminar y listar zonas. Las zonas son importantes porque dentro de ellas se organizan las atracciones del parque.

5. **Menú de atracciones:** El módulo de atracciones se encarga de administrar las atracciones pertenecientes a una zona. Cada atracción posee nombre, código, capacidad máxima, ocupación actual, estado y una fila de usuarios, este módulo permite agregar atracciones a una zona específica, buscar atracciones por código, modificar sus datos, eliminar atracciones y listar las atracciones de una zona.

6. **Menú de filas:** El módulo de filas administra la espera de usuarios para ingresar a una atracción. Permite agregar usuarios a una fila, listar la fila, buscar usuarios dentro de una fila, iniciar una vuelta de atracción y terminarla, cuando se inicia una vuelta, el sistema mueve usuarios desde la fila hacia la atracción según la capacidad máxima disponible. Al terminar la vuelta, la ocupación de la atracción vuelve a cero y se actualiza la ocupación de la zona correspondiente.

7. **Menú de reportes:** El módulo de reportes permite obtener información resumida sobre el funcionamiento del parque. Entre sus funciones se encuentran calcular ingresos totales, contar entradas vendidas, contar entradas usadas, contar visitantes totales, contar visitantes dentro del parque, mostrar ocupación de zonas y listar atracciones no operativas.

4.3. Herramientas y Tecnologías

El sistema fue desarrollado en lenguaje C, compilado con GCC. Para el desarrollo se utilizó CLion como entorno de desarrollo integrado. El control de versiones fue gestionado con Git y GitHub. Las pruebas fueron realizadas manualmente desde la terminal zsh en macOS, ejecutando el programa y navegando las opciones del menú para verificar el correcto funcionamiento de cada flujo. No se necesitó el uso de bibliotecas externas, las estructuras de datos fueran implementadas desde cero, siguiendo el material del curso, utilizando únicamente las bibliotecas estándar de C (stdio.h, stdlib.h, string.h, time.h)

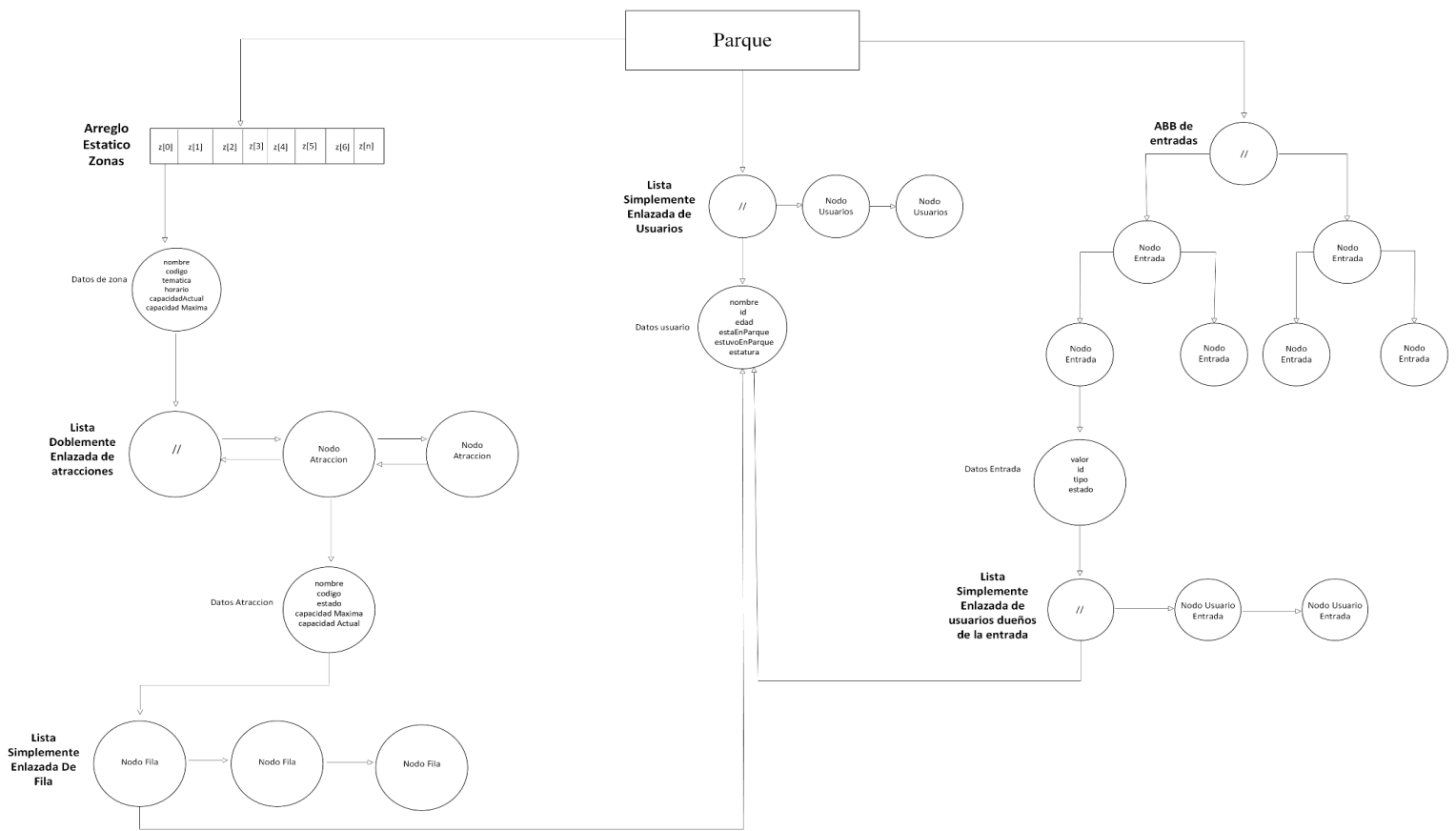
El objetivo de la solución propuesta para IBCLandia es administrar la información principal de un parque de diversiones, permitiendo gestionar usuarios, entradas, zonas, atracciones, filas y reportes.

Esta se organiza mediante una estructura principal llamada Parque, la cual centraliza los datos más importantes del sistema. Desde esta estructura se accede a las zonas del parque, la lista de usuarios registrados y el árbol de entradas. Además, el programa se divide en distintos menús para separar las funcionalidades y facilitar el uso del sistema.

4.4. Diagramas y Modelos Explicativos

Estos diagramas permiten representar gráficamente cómo se organizan los datos y cómo funcionan algunos procesos importantes del sistema.

Diagrama General de la estructura: Este diagrama realizado en Microsoft Visio muestra la organización general de los elementos del sistema. La idea es representar cómo la estructura del Parque se relaciona con las demás estructuras.



Este diagrama permite visualizar la estructura general del sistema y cómo se conectan sus principales componentes.

Diagrama de flujo de tercer nivel: El tercer nivel de anidación puede representarse como la eliminación de una atracción dentro de una zona.

Por lo tanto, el diagrama de eliminación va a mostrar el proceso para eliminar una atracción de la lista doblemente enlazada de una zona.

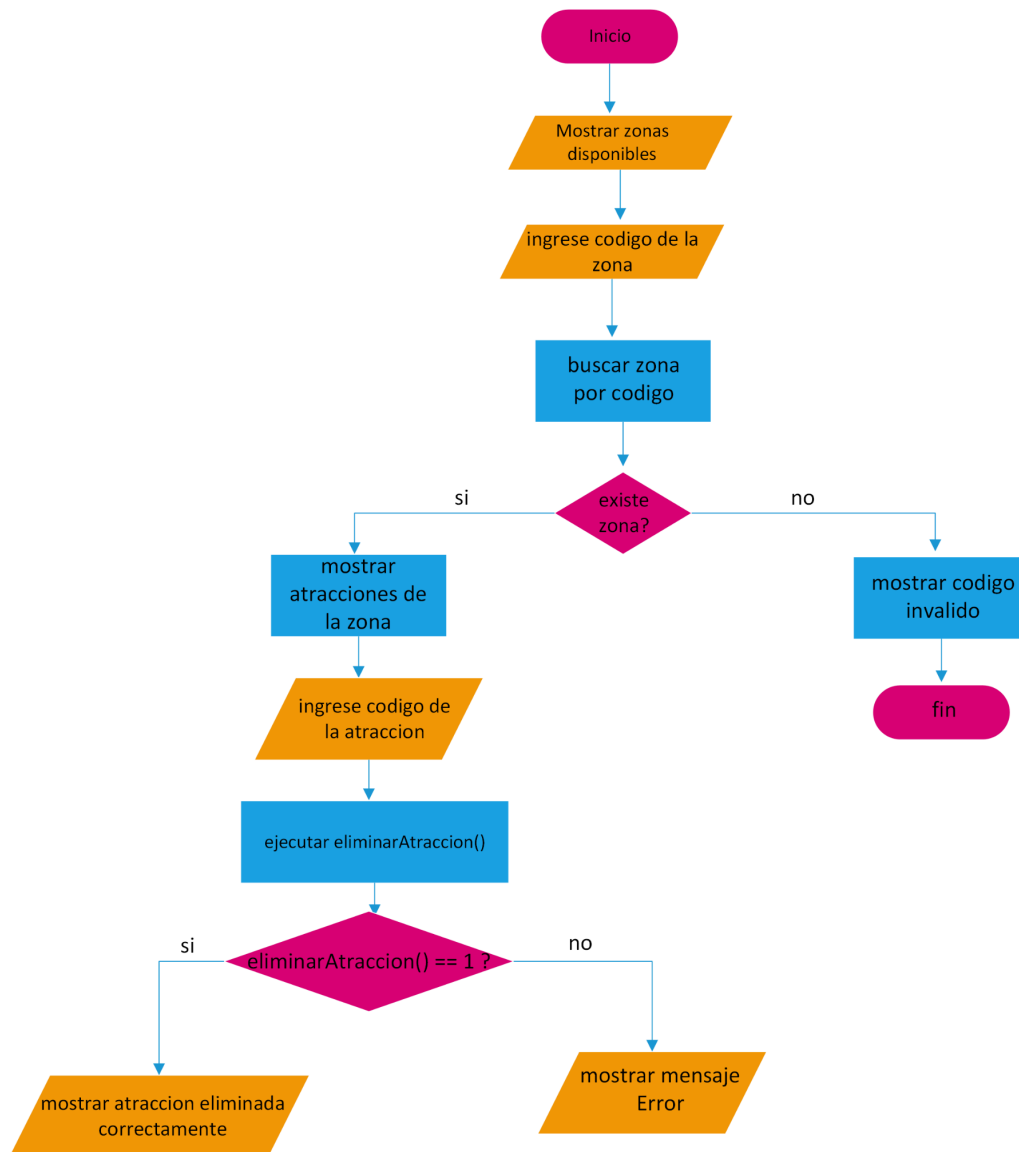
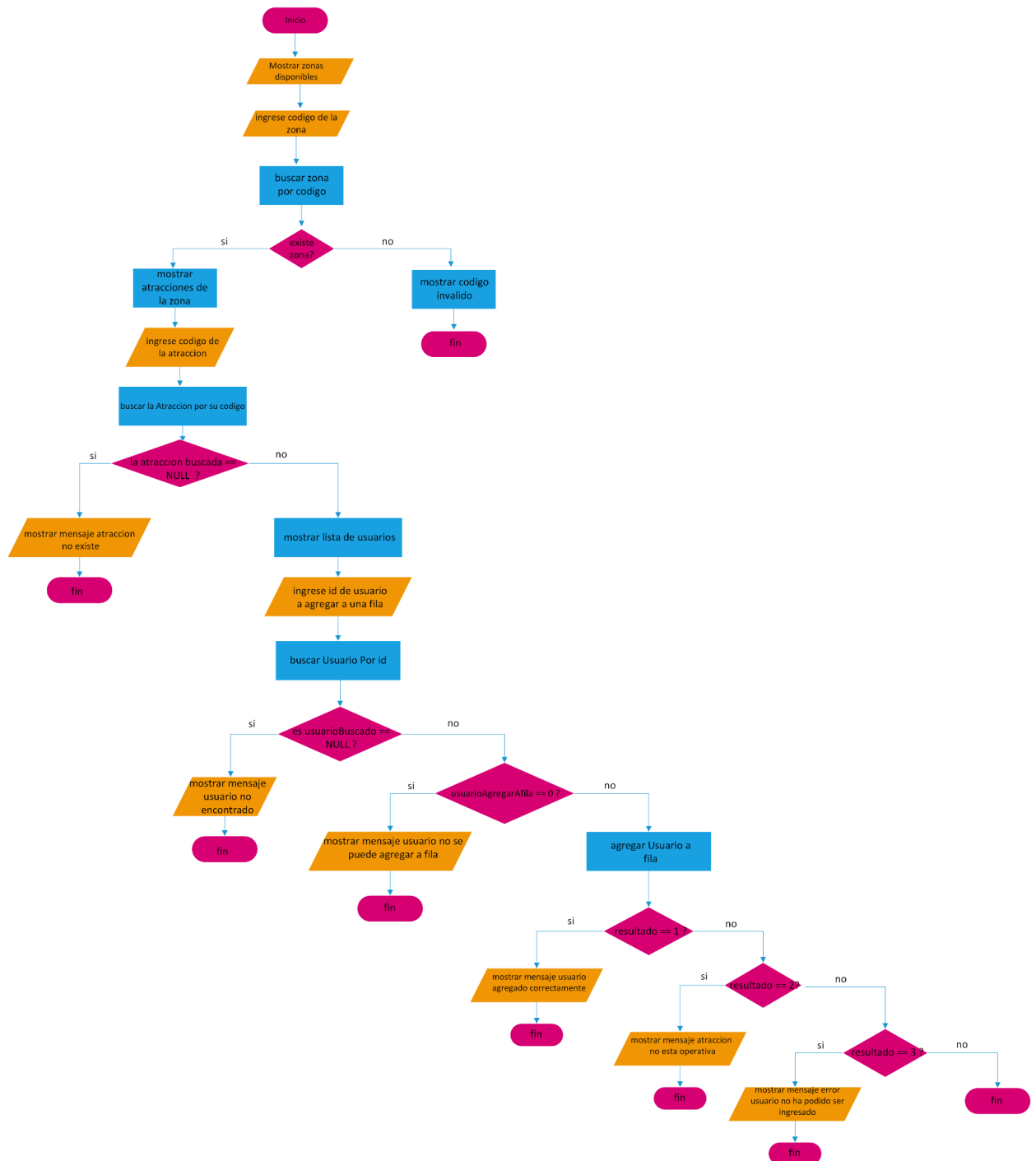


Diagrama de flujo para agregar en el último nivel: El último nivel de anidación del sistema corresponde a la fila de usuarios dentro de una atracción. Por eso, el diagrama de flujo de agregar va a representar el proceso de agregar un usuario a la fila de una atracción.

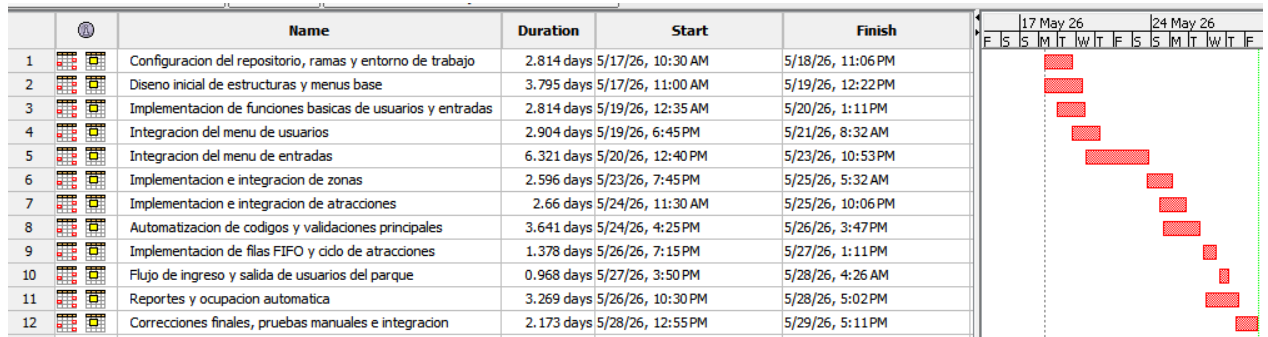


5. PLANIFICACIÓN

El proyecto fue desarrollado en un plazo de dos semanas. Donde la primera semana se dedicó al diseño de las estructuras de datos que utilizará el sistema de IBCLandia y implementación de los módulos básicos de los usuarios, zonas, parque, atracciones y entradas, completos con las funcionalidades CRUD requeridas por el sistema. En la segunda semana se implementó el módulo de filas, se agregó funcionalidad al menú con el que se opera el sistema, además de arreglar varios bugs, que impedían el correcto funcionamiento y relación entre las funciones, menús y estructuras de datos, una vez terminado el sistema se realizó una serie de pruebas simulando la correcta operación del software, finalmente se desarrolló este informe.

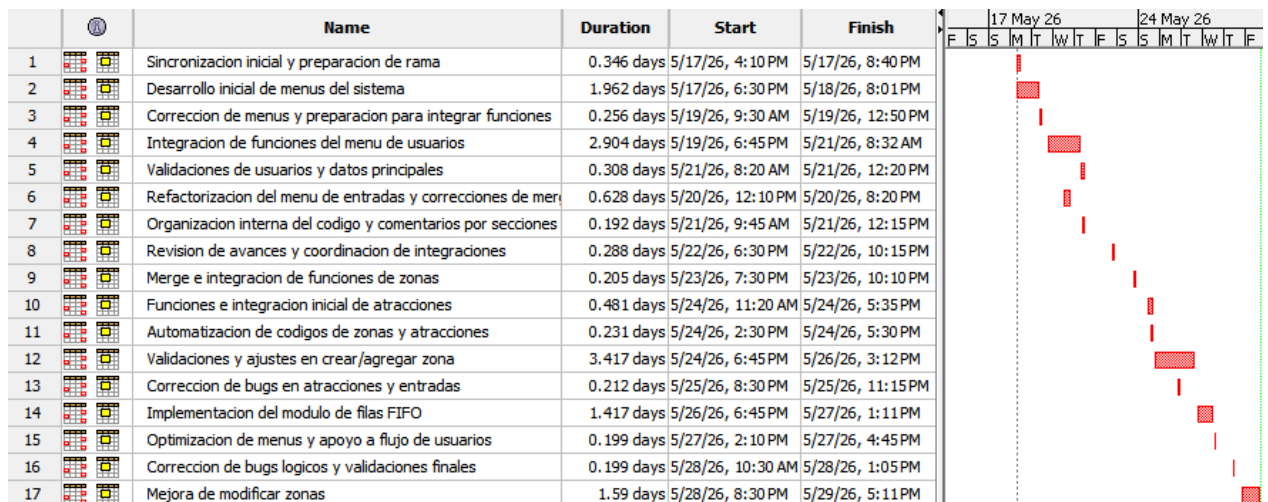
5.1. Planificación grupal

Carta de Gantt grupal:

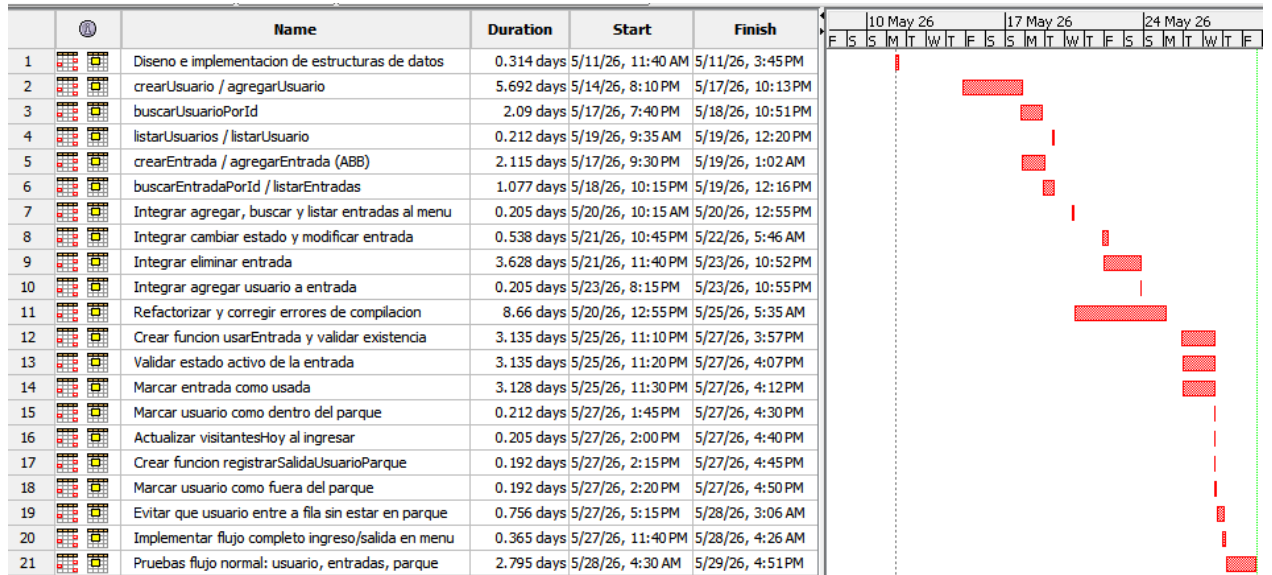


5.2. Planificación individual

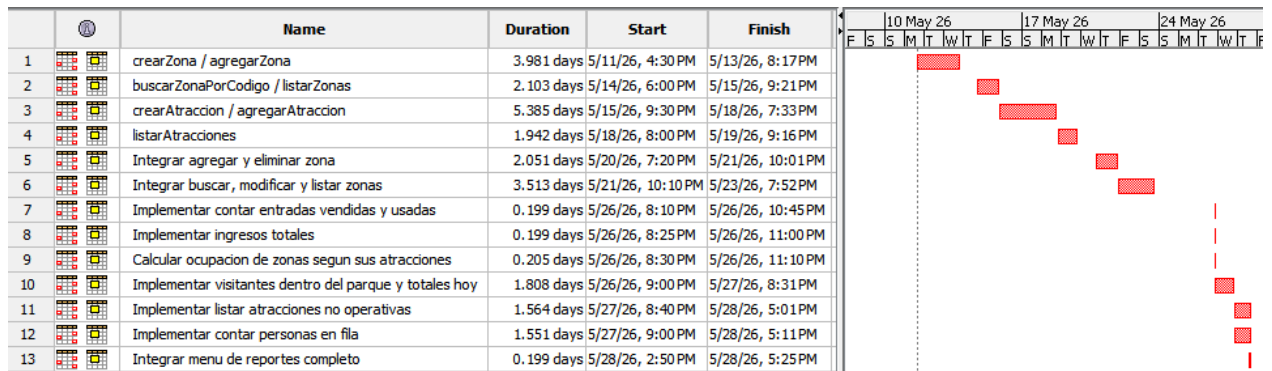
Carta de Gantt de Nicolas Jaramillo:



Carta de Gantt de Lucas Ahumada:



Carta de Gantt de Ethan Troncoso:



6. CONCLUSIÓN

El desarrollo de este proyecto permitió comprender que para abordar una solución a un problema como Ingenieros Informáticos en formación no solo dependemos de programar funciones aisladas, sino de lograr que los datos realmente se relacionen correctamente entre sí. En el caso de IBCLandia, cada elemento del sistema cumple un rol importante dentro del funcionamiento del parque: las entradas permitieron controlar el acceso de los visitantes, las zonas organizar las atracciones, las filas representan el orden de espera y los reportes nos entregan información útil sobre el estado del sistema. Por eso, uno de los aprendizajes más importantes y significativos fue entender como una mala decisión en la organización de los datos a utilizar puede dificultar posteriormente la implementación de una solución.

Durante el proceso de desarrollo también se evidenció la importancia de planificar antes de programar. Al trabajar con estructuras anidadas y punteros, fue necesario definir con claridad que estructura iba a contener a otra y desde donde debía accederse a cada dato. Esto permitió evitar una solución desordenada y facilitó la integración entre los distintos módulos del sistema. Además, el trabajar en equipo obligó a mantener una lógica común entre las funciones desarrolladas por cada integrante del equipo, sobre todo al momento de fusionar las ramas del proyecto.

La experiencia adquirida al realizar este proyecto fue valiosa, ya que permitió aplicar los contenidos vistos en la asignatura en un problema más amplio que los ejercicios realizados en clases y pruebas. A diferencia de una función puntual, este sistema exige pensar en recorridos completos, validaciones, casos de error y actualizaciones de datos después de cada operación. Esto ayudó a reforzar el uso de listas enlazadas, árboles binarios de búsqueda y arreglos, pero también permitió comprender mejor la responsabilidad asociada al diseño de un programa de diseñar un programa que debe mantenerse funcionando mientras sus datos cambian constantemente.

Si bien la solución implementada cumple con los objetivos principales del proyecto, también pueden identificarse múltiples mejoras futuras, como por ejemplo, el almacenamiento permanente de los reportes, una interfaz más amigable para el usuario o una validación más estricta de los datos ingresados. Sin embargo, se considera que, dentro del contexto de la evaluación, el sistema logra representar de manera coherente y directa un funcionamiento básico de un parque como IBCLandia y demuestra cómo las EDD pueden ser utilizadas para resolver problemas de gestión de una manera directa y efectiva.

ANEXO

El código fuente se puede encontrar en el siguiente repositorio de GitHub:
<https://github.com/The-Nick-J/IBCLandia>

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#define MAX_ZONAS 15

/* struct principal */
struct Parque
{
    struct Zona* zonas[MAX_ZONAS]; /* arreglo no dinamico de zonas */
    int cantidadZonas;
    int visitantesHoy;
    struct NodoUsuario* headUsuarios; /* lista simple enlazada de usuarios
*/
    struct NodoEntrada* raizEntradas; /* arbol binario de busqueda de
entradas */
};

struct NodoEntrada
{
    struct Entrada* datosEntrada; /* puntero a los datos de la entrada */
    struct NodoEntrada *izq, *der; /* puntero a los nodos del arbol de
entradas */
};

struct Entrada
{
    int id;
    int valor;
    int tipo; /* infantil, pase rapido,normal,familiar */ /*normal = 1,
pase rapido = 2, familiar = 3, infantil = 4*/
    int estado; /* activa, utilizada, anulada o vencida */ /*activa = 1,
utilizada = 2, anulada = 3, vencida = 4*/
    struct NodoUsuarioEntrada* headUsuarios; /* lista simplemente enlazada
de los usuarios linkeados a esta entrada */
};

struct NodoUsuarioEntrada
{
    struct Usuario* datosUsuario;
    struct NodoUsuarioEntrada* sig;
};

struct NodoUsuario
{
    struct Usuario* datosUsuario; /* puntero a los datos del usuario*/
```



```

    struct NodoUsuario* sig; /* puntero al siguiente nodo de usuarios */
};

struct Usuario
{
    char* nombre;
    int id;
    int edad;
    int estaEnParque; /* esta o no en el parque */
    int estuvoEnParque; /* estuvo o no en el parque */
    float estatura;
};

struct Zona
{
    char* nombre;
    int codigo;
    char* tematica;
    int capacidadMax;
    int ocupacionActual;
    char* horario;
    struct NodoAtraccion* headAtracciones; /* lista doblemente enlazada de
atracciones con nodo fantasma */
};

struct NodoAtraccion
{
    struct Atraccion* datosAtraccion; /* puntero a datos */
    struct NodoAtraccion *sig, *ant;
};

struct Atraccion
{
    char* nombre;
    int codigo;
    int capacidadMax;
    int ocupacionActual;
    int estado; /*en mantenimiento o funcionando */
    struct NodoFila* headFila;
};

struct NodoFila
{
    struct Usuario* datosUsuario; /* puntero a los datos del usuario */
    struct NodoFila* sig; /* puntero al siguiente nodo de la fila */
};

void inicializarParque(struct Parque* parque)
{
    int i;
    /* funcion de inicializado de parque 0 o null hasta input del
usuario*/
    parque->cantidadZonas = 0;
    parque->visitantesHoy = 0;

```

```

        parque->headUsuarios = (struct NodoUsuario*)malloc(sizeof(struct
NodoUsuario));
        if (parque->headUsuarios == NULL)
        {
            printf("error al asignar memoria\n");
            return;
        }
        parque->headUsuarios->datosUsuario = NULL;
        parque->headUsuarios->sig = NULL;

        parque->raizEntradas = (struct NodoEntrada *)malloc(sizeof(struct
NodoEntrada));

        if (parque->raizEntradas == NULL)
        {
            printf("error al asignar memoria a entradas\n");
            return;
        }

        parque->raizEntradas->datosEntrada = NULL;
        parque->raizEntradas->izq = NULL;
        parque->raizEntradas->der = NULL;

        for (i = 0; i < MAX_ZONAS; i++)
        {
            parque->zonas[i] = NULL;
        }
    }

```

```

/*funcion para crear usuarios nuevos, recibe los atributos del usuario, le
asigna memoria y retorna el puntero al usuario creado*/
struct Usuario* crearUsuario(char* nombre, int id, int edad, int
estaEnParque, int estuvoEnParque, float estatura)
{
    /*asignar memoria al nuevoUsuario y validar malloc*/
    struct Usuario* usuarioNuevo = (struct Usuario*)malloc(sizeof(struct
Usuario));
    if (usuarioNuevo == NULL)
    {
        printf("Error al asignar memoria. \n");
        return NULL;
    }
    /*asignar memoria para el nombre del nuevoUsuario y validar malloc*/
    usuarioNuevo->nombre = (char*)malloc(strlen(nombre) + 1);
    if (usuarioNuevo->nombre == NULL)
    {
        printf("Error al asignar memoria. \n");
        return NULL;
    }
    /*copiar el string de nombre y asignarlo al nuevoUsuario*/
    strcpy(usuarioNuevo->nombre, nombre);
    /*asignar valores al nuevoUsuario*/

```

```

        usuarioNuevo->id = id;
        usuarioNuevo->edad = edad;
        usuarioNuevo->estaEnParque = estaEnParque;
        usuarioNuevo->estuvoEnParque = estuvoEnParque;
        usuarioNuevo->estatura = estatura;
        return usuarioNuevo;
    }

int generarIdUsuario(void)
{
    static int ultimoIdAsignado = 0;

    ultimoIdAsignado++;
    return ultimoIdAsignado;
}

int generarIdEntrada(void)
{
    int idRandomizado;

    idRandomizado = rand() % 9000 + 1000;

    return idRandomizado;
}

int generarCodigoZona(void)
{
    static int ultimoCodigoZona = 0;
    ultimoCodigoZona++;
    return ultimoCodigoZona;
}

int generarCodigoAtraccion(void)
{
    static int ultimoCodigoAtraccion = 0;

    ultimoCodigoAtraccion++;
    return ultimoCodigoAtraccion;
}

int validarEdad(int edad)
{
    if (edad < 1 || edad > 120)
    {
        return 0;
    }
    return 1;
}

int validarEstatura(float estatura)
{
    if (estatura < 0.5 || estatura > 2.5)
    {

```

```

        return 0;
    }
    return 1;
}

int validarNombreCompleto(char* nombre)
{
    int i = 0;
    int letrasAntes = 0;
    int letrasDespues = 0;
    int espacio = 0;

    if (nombre == NULL) return 0;

    while (nombre[i] != '\0')
    {
        if (nombre[i] == ' ')
        {
            if (letrasAntes >= 4)
            {
                espacio = 1;
            }
        }
        else
        {
            if (espacio == 0)
            {
                letrasAntes++;
            }
            else
            {
                letrasDespues++;
            }
        }
        i++;
    }
    if (letrasAntes >= 4 && espacio == 1 && letrasDespues >= 4)
    {
        return 1;
    }
    return 0;
}

int validarHorarioZona(char* horario)
{
    if (horario == NULL)
    {
        return 0;
    }

    if (strlen(horario) != 11)
    {
        return 0;
    }
}

```

```

        if (horario[2] != ':' || horario[5] != '-' || horario[8] != ':')
        {
            return 0;
        }
        return 1;
    }

int validarTextoZona(char* texto)
{
    if (texto == NULL)
    {
        return 0;
    }

    if (strlen(texto) < 3)
    {
        return 0;
    }

    return 1;
}

/*funcion auxiliar para reemplazar nodos en un arbol binario de busqueda*/
void reemplazarNodoABB(struct NodoEntrada** entradas, struct NodoEntrada**
nodo)
{
    if (!(*entradas)->der)
    {
        (*nodo)->datosEntrada = (*entradas)->datosEntrada;
        *nodo = *entradas;
        *entradas = (*entradas)->izq;
    }
    else
    {
        reemplazarNodoABB(&(*entradas)->der, &(*nodo));
    }
}

struct Usuario* leerDatosCrearUsuario(void)
{
    char nombre[100];
    int id;
    int edad;
    int estaEnParque;
    int estuvoEnParque;
    float estatura;

    printf("\nBienvenido al menu de creacion de usuario!\n");

    do
    {
        printf("Ingrese el nombre del usuario (Nombre Apellido): ");
        scanf("%99[^\n]", nombre);
    }

```

```

        if (validarNombreCompleto(nombre) == 0)
        {
            printf(
                "Nombre invalido, debe ingresar un nombre separado por
espacio, nombre y apellido con 4 letras minimo cada uno.\n");
        }
    }
    while (validarNombreCompleto(nombre) == 0);

do
{
    printf("Ingrese la edad del usuario: ");
    scanf(" %d", &edad);

    if (validarEdad(edad) == 0)
    {
        printf("Edad invalida. Intente nuevamente.\n");
    }
}
while (validarEdad(edad) == 0);

do
{
    printf("Ingrese la estatura del usuario ej:(1.78) ");
    scanf(" %f", &estatura);

    if (validarEstatura(estatura) == 0)
    {
        printf("Estatura no valida. debe estar entre 0.5m y 2.5m\n");
    }
}
while (validarEstatura(estatura) == 0);

id = generarIdUsuario();
estaEnParque = 0;
estuvoEnParque = 0;

printf("Usuario fue creado con ID: %d\n", id);

    return crearUsuario(nombre, id, edad, estaEnParque, estuvoEnParque,
estatura);
}

/*funcion para crear entradas nuevas, recibe los atributos de la entrada,
le asigna memoria y retorna el puntero a la entrada*/
struct Entrada* crearEntrada(int valor, int tipo, int estado)
{
    /*asignar memoria a la nuevaEntrada y validar malloc*/
    struct Entrada* entradaNueva = (struct Entrada*)malloc(sizeof(struct
Entrada));
    if (entradaNueva == NULL)
    {

```

```

        printf("Error al asignar memoria \n");
        return NULL;
    }

    /*asignar valores a la nuevaEntrada*/
    entradaNueva->id = generarIdEntrada();
    entradaNueva->valor = valor;
    entradaNueva->tipo = tipo;
    entradaNueva->estado = estado;
    entradaNueva->headUsuarios = NULL;

    entradaNueva->headUsuarios = (struct
    NodoUsuarioEntrada*)malloc(sizeof(struct NodoUsuarioEntrada));
    if (entradaNueva->headUsuarios == NULL)
    {
        printf("error al asignar memoria\n");
        return NULL;
    }

    entradaNueva->headUsuarios->datosUsuario = NULL;
    entradaNueva->headUsuarios->sig = NULL;

    return entradaNueva;
}

/*funcion para eliminar entradas del arbol binario de busqueda de
entradas*/
void eliminarEntrada(struct NodoEntrada** entradas, int id)
{
    struct NodoEntrada* nodo = NULL;
    if (!(*entradas)) { return; }
    if ((*entradas)->datosEntrada->id < id)
    {
        eliminarEntrada(&(*entradas)->der, id);
    }
    else
    {
        if ((*entradas)->datosEntrada->id > id)
        {
            eliminarEntrada(&(*entradas)->izq, id);
        }
        else
        {
            if ((*entradas)->datosEntrada->id == id)
            {
                nodo = *entradas;
                if (!(*entradas)->izq)
                {
                    *entradas = (*entradas)->der;
                }
                else
                {
                    if (!(*entradas)->der)
                    {

```

```

        *entradas = (*entradas)->izq;
    }
    else
    {
        reemplazarNodoABB(&(*entradas)->izq, &nodo);
    }
}
}
}

/*funcion para agregar un usuario al ultimo nodo de usuarios*/
int agregarUsuario(struct Parque* parque, struct Usuario* usuario)
{
    /*crear el nuevo nodo al que va conectado con el nuevo usuario y
    validar malloc*/
    struct NodoUsuario* nuevoNodo;
    struct NodoUsuario* rec;

    nuevoNodo = (struct NodoUsuario*)malloc(sizeof(struct NodoUsuario));
    if (nuevoNodo == NULL)
    {
        printf("Error al asignar memoria. \n");
        return 0;
    }
    /*asignar usuario al nuevo nodo de usuario*/
    nuevoNodo->datosUsuario = usuario;
    nuevoNodo->sig = NULL;
    /*rec para iterar por la lista simple*/
    rec = parque->headUsuarios;
    /*inserta el nodo de usuario en el final*/
    rec = parque->headUsuarios;

    while (rec->sig != NULL)
    {
        rec = rec->sig;
    }

    rec->sig = nuevoNodo;
    return 1;
}

/*insertar la entrada de forma recursiva al ABB (falta comentar, toy
cansado me voy a acostar lol)*/
struct NodoEntrada* agregarEntrada(struct NodoEntrada* arbol, struct
Entrada* entrada)
{
    if (arbol == NULL)
    {
        arbol = (struct NodoEntrada*)malloc(sizeof(struct NodoEntrada));
        arbol->datosEntrada = entrada;
        arbol->izq = NULL;
        arbol->der = NULL;
    }
}

```



```

        return arbol;
    }

    if (entrada->id < arbol->datosEntrada->id)
    {
        arbol->izq = agregarEntrada(arbol->izq, entrada);
    }
    else
    {
        if (entrada->id > arbol->datosEntrada->id)
        {
            arbol->der = agregarEntrada(arbol->der, entrada);
        }
    }
    return arbol;
}

/*funcion para buscar usuarios por id*/
struct Usuario* buscarUsuarioPorId(struct Parque* parque, int idUsuario)
{
    struct Usuario* buscado = NULL;
    struct NodoUsuario* rec = parque->headUsuarios->sig;
    while (rec != NULL)
    {
        if (rec->datosUsuario->id == idUsuario)
        {
            buscado = rec->datosUsuario;
            return buscado;
        }
        rec = rec->sig;
    }
    return NULL;
}

int eliminarUsuario(struct Parque* parque, int idUsuario)
{
    struct NodoUsuario* actual;
    struct NodoUsuario* anterior;

    anterior = parque->headUsuarios;
    actual = parque->headUsuarios->sig;

    while (actual != NULL)
    {
        if (actual->datosUsuario->id == idUsuario)
        {
            anterior->sig = actual->sig;

            return 1;
        }
        anterior = actual;
        actual = actual->sig;
    }
}

```

```

    }
    return 0;
}

int  modificarUsuario(struct  Parque*  parque,  int  idUsuario,  char*
nombreNuevo, int edadNueva, float estaturaNueva)
{
    struct Usuario* usuarioCambiar;

    usuarioCambiar = buscarUsuarioPorId(parque, idUsuario);

    if (usuarioCambiar == NULL) return 0;

    usuarioCambiar->nombre = (char*)malloc(strlen(nombreNuevo) + 1);

    strcpy(usuarioCambiar->nombre, nombreNuevo);
    usuarioCambiar->edad = edadNueva;
    usuarioCambiar->estatura = estaturaNueva;

    return 1;
}

struct  Entrada*  buscarEntradaPorId(struct  NodoEntrada*  arbol,  int
idEntrada)
{
    if (arbol == NULL) return NULL;

    if (arbol->datosEntrada->id == idEntrada)
    {
        return arbol->datosEntrada;
    }

    if (arbol->datosEntrada->id > idEntrada)
    {
        return buscarEntradaPorId(arbol->izq, idEntrada);
    }
    else
    {
        return buscarEntradaPorId(arbol->der, idEntrada);
    }
}

/*funcion para listar usuario individual, recibe struct del usuario que se
lista*/
void listarUsuario(struct Usuario* usuario)
{
    printf("Nombre: %s\n", usuario->nombre);
    printf("Id: %d\n", usuario->id);
    printf("| Edad: %d\n", usuario->edad);
    printf("| Estatura: %.2f\n", usuario->estatura);
    printf("-----\n");
}

```

```

/*funcion para listar todos los usuarios del sistema, recibe el struct
principal*/
void listarTodosLosUsuarios(struct Parque* parque)
{
    int i = 0;
    struct NodoUsuario* rec = parque->headUsuarios->sig;
    while (rec != NULL)
    {
        printf("\nUsuario %d\n: ", i);
        listarUsuario(rec->datosUsuario);
        i++;
        rec = rec->sig;
    }
}

void listarTodosLosUsuarioSimple(struct Parque* parque)
{
    struct NodoUsuario* rec;

    if (parque->headUsuarios->sig == NULL)
    {
        printf("no hay usuarios registrados \n");
        return;
    }

    printf("usuarios disponibles: \n");

    rec = parque->headUsuarios->sig;

    while (rec != NULL)
    {
        printf("id:  %d  |  nombre:  %s\n", rec->datosUsuario->id,
rec->datosUsuario->nombre);
        rec = rec->sig;
    }
}

/*funcion auxiliar para pasar el tipo de entrada de int a string*/
char* obtenerTipoEntrada(int tipo)
{
    if (tipo == 1)
    {
        return "Normal";
    }
    if (tipo == 2)
    {
        return "Pase Rapido";
    }
    if (tipo == 3)
    {
        return "Familiar";
    }
    if (tipo == 4)
    {

```

```

        return "Infantil";
    }
    return "Error: tipo de entrada invalido";
}

/*funcion auxiliar para pasar el estado de entrada de int a string*/
char* obtenerEstadoEntrada(int estado)
{
    if (estado == 1)
    {
        return "Activa";
    }
    if (estado == 2)
    {
        return "Utilizada";
    }
    if (estado == 3)
    {
        return "Anulada";
    }
    if (estado == 4)
    {
        return "Vencida";
    }
    return "Error: estado de entrada invalido";
}

/* funcion para listar entradas de forma individual*/
void listarEntrada(struct Entrada *entrada)
{
    printf("| Id : %d\n", entrada->id);
    printf("| Valor : %d\n", entrada->valor);
    printf("| Tipo : %s\n", obtenerTipoEntrada(entrada->tipo));
    printf("| Estado : %s\n", obtenerEstadoEntrada(entrada->estado));
    printf("-----\n");
    struct NodoUsuarioEntrada* rec = entrada->headUsuarios->sig;
    if (rec == NULL)
    {
        printf("no hay usuarios asociados a esta entrada\n");
        printf("-----\n");
        return;
    }
    printf("Usuarios asociados a esta entrada: \n");
    while (rec != NULL)
    {
        printf("Id:  %d | Nombre:  %s\n", rec->datosUsuario->id,
rec->datosUsuario->nombre);
        rec = rec->sig;
    }
    printf("-----\n");
}

/*funcion para listar TODAS las entradas con recorrido INORDER por el
ABB*/

```

```

void listarTodasLasEntradas(struct NodoEntrada* arbol)
{
    if (arbol)
    {
        listarTodasLasEntradas(arbol->izq);
        printf("\nEntrada : ");
        listarEntrada(arbol->datosEntrada);
        listarTodasLasEntradas(arbol->der);
    }
}

/*atracciones y zonas*/
struct Zona* buscarZonaPorCodigo(struct Parque* parque, int codigoBuscado)
{
    int i;
    if (parque == NULL)
    {
        return NULL;
    }
    for (i = 0; i < parque->cantidadZonas; i++)
    {
        if (parque->zonas[i] != NULL && parque->zonas[i]->codigo ==
codigoBuscado)
        {
            return parque->zonas[i];
        }
    }

    return NULL;
}

void mostrarZona(struct Zona* zona, struct Parque* parque)
{
    int i;
    if (zona == NULL)
    {
        printf("Zona invalida\n");
        return;
    }
    for (i = 0; i < parque->cantidadZonas; i++)
    {
        if (parque->zonas[i] == zona)
        {
            printf("nombre: %s\n", zona->nombre);
            printf("codigo: %d\n", zona->codigo);
            printf("horario: %s\n", zona->horario);
            printf("tematica: %s\n", zona->tematica);
            printf("ocupacion actual: %d\n", zona->ocupacionActual);
            printf("capacidad maxima: %d\n", zona->capacidadMax);
            printf("-----\n");
            return;
        }
    }
    return;
}

```

```

}

void compactarZonas(struct Parque* parque, int indice)
{
    int j;

    if (parque == NULL)
    {
        return;
    }

    for (j = indice; j < parque->cantidadZonas - 1; j++)
    {
        parque->zonas[j] = parque->zonas[j + 1];
    }

    parque->zonas[parque->cantidadZonas - 1] = NULL;
}

void liberarZona(struct Zona* zona)
{
    if (zona == NULL)
    {
        return;
    }
}

struct Atraccion* crearAtraccion(void)
{
    struct Atraccion* nuevaAtraccion;
    char nombre[100];

    nuevaAtraccion = (struct Atraccion*)malloc(sizeof(struct Atraccion));
    if (nuevaAtraccion == NULL)
    {
        printf("error al asignar memoria\n");
        return NULL;
    }

    printf("ingrese el nombre de la atraccion: ");
    scanf(" %99[^\n]", nombre);

    nuevaAtraccion->nombre = (char*)malloc(strlen(nombre) + 1);
    if (nuevaAtraccion->nombre == NULL)
    {
        printf("error al asignar memoria\n");
        return NULL;
    }

    strcpy(nuevaAtraccion->nombre, nombre);

    nuevaAtraccion->codigo = generarCodigoAtraccion();
    printf("codigo de la atraccion ha sido generado: %d\n",
nuevaAtraccion->codigo);
}

```

```

do
{
    printf("ingrese la capacidad maxima de la atraccion a crear: ");
    scanf(" %d", &nuevaAtraccion->capacidadMax);

    if (nuevaAtraccion->capacidadMax <= 0)
    {
        printf("la capacidad debe ser mayor a 0\n");
    }
}
while (nuevaAtraccion->capacidadMax <= 0);

nuevaAtraccion->ocupacionActual = 0;
nuevaAtraccion->estado = 1;
nuevaAtraccion->headFila = NULL;

return nuevaAtraccion;
}

void listarAtracciones(struct NodoAtraccion* headAtracciones)
{
    struct NodoAtraccion* rec;
    int i = 1;

    if (headAtracciones == NULL)
    {
        return;
    }

    rec = headAtracciones->sig;

    while (rec != NULL)
    {
        printf("Atraccion %d\n", i);
        printf("Nombre: %s\n", rec->datosAtraccion->nombre);
        printf("Codigo: %d\n", rec->datosAtraccion->codigo);
        printf("Capacidad      maxima:      %d\n",
rec->datosAtraccion->capacidadMax);
        printf("Ocupacion      actual:      %d\n",
rec->datosAtraccion->ocupacionActual);
        printf("Estado: %d\n", rec->datosAtraccion->estado);
        printf("-----\n");
        i++;
        rec = rec->sig;
    }
}

void listarAtraccionesSimple(struct NodoAtraccion* headAtracciones)
{
    struct NodoAtraccion* rec;

```

```

    rec = headAtracciones->sig;

    if (rec == NULL)
    {
        printf("no hay atracciones en esta zona\n");
        return;
    }

    printf("atracciones disponibles: \n");

    while (rec != NULL)
    {
        printf("codigo: %d | nombre: %s\n", rec->datosAtraccion->codigo,
rec->datosAtraccion->nombre);
        rec = rec->sig;
    }
}

struct Atraccion* buscarAtraccionPorCodigo(struct NodoAtraccion*
headAtracciones, int codigoAtraccion)
{
    struct NodoAtraccion* rec;

    if (headAtracciones == NULL) return NULL;

    rec = headAtracciones->sig;

    while (rec != NULL)
    {
        if (rec->datosAtraccion->codigo == codigoAtraccion)
        {
            return rec->datosAtraccion;
        }

        rec = rec->sig;
    }
    return NULL;
}

int agregarUsuarioAFila(struct Atraccion* atraccion, struct Usuario*
usuario)
{
    struct NodoFila* nuevoNodo;
    struct NodoFila* rec;

    if (atraccion == NULL || usuario == NULL) return 0;

    if (atraccion->estado != 1) return 2;

    nuevoNodo = (struct NodoFila*)malloc(sizeof(struct NodoFila));

    if (nuevoNodo == NULL) return 0;

    nuevoNodo->datosUsuario = usuario;

```



```

nuevoNodo->sig = NULL;

if (atraccion->headFila == NULL)
{
    atraccion->headFila = nuevoNodo;
    return 1;
}

rec = atraccion->headFila;

while (rec->sig != NULL)
{
    rec = rec->sig;
}

rec->sig = nuevoNodo;
return 1;
}

void listarFilaAtraccion(struct Atraccion* atraccion)
{
    struct NodoFila* rec;
    int i;

    if (atraccion->headFila == NULL)
    {
        printf("la fila de esta atraccion esta vacia\n");
        return;
    }

    rec = atraccion->headFila;
    i = 1;

    while (rec != NULL)
    {
        printf("persona %d en fila\n", i);
        printf("nombre: %s\n", rec->datosUsuario->nombre);
        printf("id: %d\n", rec->datosUsuario->id);
        printf("edad: %d\n", rec->datosUsuario->edad);
        printf("estatura: %.2f\n", rec->datosUsuario->estatura);
        printf("-----\n");

        i++;
        rec = rec->sig;
    }
}

struct Usuario* buscarUsuarioEnFila(struct Atraccion* atraccion, int id)
{
    struct NodoFila* rec;

    rec = atraccion->headFila;

```

```

while (rec != NULL)
{
    if (rec->datosUsuario->id == id)
    {
        return rec->datosUsuario;
    }
    rec = rec->sig;
}
return NULL;
}

int quitarUsuarioDeFila(struct Atraccion* atraccion, int idUsuario)
{
    struct NodoFila* actual;
    struct NodoFila* anterior = NULL;

    actual = atraccion->headFila;

    while (actual != NULL)
    {
        if (actual->datosUsuario->id == idUsuario)
        {
            if (anterior == NULL)
            {
                atraccion->headFila = actual->sig;
            }
            else
            {
                anterior->sig = actual->sig;
            }
            return 1;
        }
        anterior = actual;
        actual = actual->sig;
    }
    return 0;
}

void vaciarFilaAtraccion(struct Atraccion* atraccion)
{
    atraccion->headFila = NULL;
}

struct Atraccion* buscarUsuarioEnTodasLasFilas(struct Parque* parque, int id)
{
    int i;
    struct NodoAtraccion* rec;

    for (i = 0; i < parque->cantidadZonas; i++)
    {
        rec = parque->zonas[i]->headAtracciones->sig;
    }
}

```

```

        while (rec != NULL)
        {
            if (buscarUsuarioEnFila(rec->datosAtraccion, id) != NULL)
            {
                return rec->datosAtraccion;
            }
            rec = rec->sig;
        }
    }
    return NULL;
}

int validarUsuarioSiPuedeFila(struct Parque* parque, struct Atraccion*
atraccion, int id)
{
    struct Atraccion* dondeEstaUsuario;
    int opcionMover;
    struct Usuario* usuario = buscarUsuarioPorId(parque, id);

    if (usuario == NULL)
    {
        printf("usuario no existe\n");
        return 0;
    }

    if (usuario->estaEnParque == 0)
    {
        printf("El usuario no esta dentro del parque\n");
        return 0;
    }

    dondeEstaUsuario = buscarUsuarioEnTodasLasFilas(parque, id);

    if (dondeEstaUsuario == NULL)
    {
        return 1;
    }

    if (dondeEstaUsuario == atraccion)
    {
        printf("este usuario ya esta en la fila de esta atraccion\n");
        return 0;
    }

    printf("este usuario ya esta en otra fila\n");
    printf("desea quitarlo y moverlo a esta fila?\n");
    printf("1. si\n");
    printf("2. no\n");
    scanf(" %d", &opcionMover);

    if (opcionMover != 1)
    {
        printf("usuario NO fue agregado a la nueva fila\n");
    }
}

```

```

        return 0;
    }

    quitarUsuarioDeFila(dondeEstaUsuario, id);
    return 1;
}

int iniciarVueltaAtraccion(struct Atraccion* atraccion)
{
    struct NodoFila* actual;
    int cantidadSubidos = 0;

    if (atraccion == NULL)
    {
        return -3;
    }

    if (atraccion->estado != 1)
    {
        return -1;
    }

    if (atraccion->ocupacionActual > 0)
    {
        return -2;
    }

    if (atraccion->headFila == NULL)
    {
        return 0;
    }

    while (atraccion->headFila != NULL && cantidadSubidos <
atraccion->capacidadMax)
    {
        actual = atraccion->headFila;
        atraccion->headFila = actual->sig;

        cantidadSubidos++;
        atraccion->ocupacionActual++;
    }

    return cantidadSubidos;
}

int terminarVueltaAtraccion(struct Atraccion* atraccion)
{
    int personasQueSeVan;

    if (atraccion->ocupacionActual == 0)
    {
        return 0;
    }
}

```

```

    personasQueSeVan = atraccion->ocupacionActual;
    atraccion->ocupacionActual = 0;

    return personasQueSeVan;
}

int eliminarAtraccion(struct NodoAtraccion* headAtracciones, int
codigoAtraccion)
{
    struct NodoAtraccion* actual;

    if (headAtracciones == NULL) return 0;

    actual = headAtracciones->sig;

    while (actual != NULL)
    {
        if (actual->datosAtraccion->codigo == codigoAtraccion)
        {
            actual->ant->sig = actual->sig;

            if (actual->sig != NULL)
            {
                actual->sig->ant = actual->ant;
            }

            return 1;
        }
        actual = actual->sig;
    }
    return 0;
}

void cambiarNombreAtraccion(struct Atraccion* atraccion, char*
nombreNuevo)
{
    if (atraccion == NULL || nombreNuevo == NULL) return;

    atraccion->nombre = (char*)malloc(strlen(nombreNuevo) + 1);

    if (atraccion->nombre == NULL) return;

    strcpy(atraccion->nombre, nombreNuevo);
}

void cambiarCapacidadAtraccion(struct Atraccion* atraccion, int
capacidadNueva)
{
    if (atraccion == NULL) return;
    atraccion->capacidadMax = capacidadNueva;
}

void cambiarEstadoAtraccion(struct Atraccion* atraccion, int estadoNuevo)

```

```

{
    if (atraccion == NULL) return;

    atraccion->estado = estadoNuevo;

    if (estadoNuevo != 1)
    {
        vaciarFilaAtraccion(atraccion);
        atraccion->ocupacionActual = 0;
    }
}

void modificarAtraccion(struct Atraccion* atraccion, int opcion)
{
    char nombreNuevo[100];
    int valorNuevo;

    if (atraccion == NULL) return;

    if (opcion == 1)
    {
        printf("ingrese el nuevo nombre de la atraccion: ");
        scanf(" %99[^\n]", nombreNuevo);
        cambiarNombreAtraccion(atraccion, nombreNuevo);
    }
    else if (opcion == 2)
    {
        do
        {
            printf("ingrese la nueva capacidad maxima: ");
            scanf(" %d", &valorNuevo);

            if (valorNuevo <= 0)
            {
                printf("capacidad invalida, debe ser mayor a 0\n");
            }
        }
        while (valorNuevo <= 0);
        cambiarCapacidadAtraccion(atraccion, valorNuevo);
    }
    else if (opcion == 3)
    {
        do
        {
            printf("Ingrese el nuevo estado:\n");
            printf("1. Operativa\n");
            printf("2. En mantenimiento\n");
            printf("3. Fuera de servicio\n");
            printf("4. Cerrada por horario\n");
            scanf(" %d", &valorNuevo);

            if (valorNuevo < 1 || valorNuevo > 4)
            {

```

```

        printf("estado invalido, ingrese una opcion entre 1 y
4\n");
    }
    }
    while (valorNuevo < 1 || valorNuevo > 4);
    cambiarEstadoAtraccion(atraccion, valorNuevo);
}

int eliminarZona(struct Parque* parque, struct Zona* zona)
{
    int i;
    struct Zona* zonaBorrar;

    if (parque == NULL || zona == NULL)
    {
        return 0;
    }

    for (i = 0; i < parque->cantidadZonas; i++)
    {
        if (parque->zonas[i] == zona)
        {
            zonaBorrar = parque->zonas[i];

            compactarZonas(parque, i);
            parque->cantidadZonas--;
            liberarZona(zonaBorrar);
            return 1;
        }
    }
    return 0;
}

int cambiarNombreZona(struct Zona *zona, char *nombreNuevo)
{
    zona->nombre = (char*)malloc((strlen(nombreNuevo) + 1) *
sizeof(char));

    if (zona->nombre == NULL) return 0;

    strcpy(zona->nombre, nombreNuevo);
    return 1;
}

int cambiarTematicaZona(struct Zona *zona, char *tematicaNueva)
{
    zona->tematica = (char *)malloc((strlen(tematicaNueva) + 1) *
sizeof(char));

    if (zona->tematica == NULL) return 0;

    strcpy(zona->tematica, tematicaNueva);
}

```

```

        return 1;
    }

int cambiarHorarioZona(struct Zona *zona, char *horarioNuevo)
{
    zona->horario = (char *)malloc((strlen(horarioNuevo) + 1) *
sizeof(char));

    if (zona->horario == NULL) return 0;

    strcpy(zona->horario, horarioNuevo);
    return 1;
}

int cambiarCapacidadZona(struct Zona *zona, int capacidadMaxima)
{
    zona->capacidadMax = capacidadMaxima;
    return 1;
}

void agregarAtraccion(struct NodoAtraccion* headAtracciones)
{
    struct NodoAtraccion* rec;
    struct Atraccion* nuevaAtraccion;
    struct NodoAtraccion* nuevoNodo;

    if (headAtracciones == NULL)
    {
        return;
    }

    nuevaAtraccion = crearAtraccion();
    if (nuevaAtraccion == NULL)
    {
        printf("Fallo al crear nueva Atraccion !\n");
        return;
    }

    nuevoNodo = (struct NodoAtraccion*)malloc(sizeof(struct
NodoAtraccion));
    if (nuevoNodo == NULL)
    {
        return;
    }

    nuevoNodo->datosAtraccion = nuevaAtraccion;
    nuevoNodo->sig = NULL;
    nuevoNodo->ant = NULL;

    rec = (headAtracciones)->sig;
    /*Caso que solo exista el nodo fantasma*/
    if (rec == NULL)
    {

```



```

        (headAtracciones)->sig = nuevoNodo;
        nuevoNodo->ant = (headAtracciones);
        printf("Atraccion agregada correctamente \n");
        return;
    }

    while (rec != NULL)
    {
        if (rec->sig == NULL)
        {
            nuevoNodo->ant = rec;
            rec->sig = nuevoNodo;
            printf("Atraccion agregada correctamente \n");
            return;
        }
        rec = rec->sig;
    }
}

void listarZonas(struct Parque* parque)
{
    int i;

    if (parque == NULL)
        return;

    for (i = 0; i < parque->cantidadZonas; i++)
    {
        printf("Zona %d\n", i + 1);
        printf("Nombre: %s\n", parque->zonas[i]->nombre);
        printf("Codigo: %d\n", parque->zonas[i]->codigo);
        printf("Horario: %s\n", parque->zonas[i]->horario);
        printf("Tematica: %s\n", parque->zonas[i]->tematica);
        printf("Ocupacion actual: %d\n",
parque->zonas[i]->ocupacionActual);
        printf("Capacidad maxima: %d\n", parque->zonas[i]->capacidadMax);
        printf("-----\n");
    }
}

void listarZonasSimple(struct Parque* parque)
{
    int i;

    if (parque->cantidadZonas == 0)
    {
        printf("No hay zonas\n");
        return;
    }

    printf("Zonas disponibles: \n");

    for (i = 0; i < parque->cantidadZonas; i++)
    {

```

```

        printf("Codigo: %d | Nombre: %s\n", parque->zonas[i]->codigo,
parque->zonas[i]->nombre);
    }
}

struct Zona* crearZona(char* nombre, int codigo, char* tematica, int
capacidadMax, int ocupacionActual, char* horario)
{
    struct Zona* zonaNueva;
    struct NodoAtraccion* headAtracciones;

    zonaNueva = (struct Zona*)malloc(sizeof(struct Zona));

    if (zonaNueva == NULL)
    {
        printf("error al crear zona\n");
        return NULL;
    }

    zonaNueva->nombre = (char*)malloc(strlen(nombre) + 1);
    if (zonaNueva->nombre == NULL)
    {
        return NULL;
    }

    strcpy(zonaNueva->nombre, nombre);

    zonaNueva->tematica = (char*)malloc(strlen(tematica) + 1);
    if (zonaNueva->tematica == NULL)
    {
        return NULL;
    }

    strcpy(zonaNueva->tematica, tematica);

    zonaNueva->horario = (char*)malloc(strlen(horario) + 1);
    if (zonaNueva->horario == NULL)
    {
        return NULL;
    }

    strcpy(zonaNueva->horario, horario);

    headAtracciones = (struct NodoAtraccion*)malloc(sizeof(struct
NodoAtraccion));

    if (headAtracciones == NULL)
    {
        return NULL;
    }

    headAtracciones->datosAtraccion = NULL;
    headAtracciones->sig = NULL;
    headAtracciones->ant = NULL;

```

```

    zonaNueva->codigo = codigo;
    zonaNueva->capacidadMax = capacidadMax;
    zonaNueva->ocupacionActual = ocupacionActual;
    zonaNueva->headAtracciones = headAtracciones;

    return zonaNueva;
}

void agregarZona(struct Parque* parque)
{
    char nombre[100];
    char tematica[100];
    char horario[100];
    int codigo;
    int capacidadMax;
    struct Zona* nuevaZona;

    if (parque == NULL)
    {
        printf("parque no existe\n");
        return;
    }

    if (parque->cantidadZonas >= MAX_ZONAS)
    {
        printf("no se pueden agregar mas zonas\n");
        return;
    }

    do
    {
        printf("ingrese el nombre de la zona: ");
        scanf(" %99[^\n]", nombre);
        if (validarTextoZona(nombre) == 0)
        {
            printf("nombre invalido\n");
        }
    }
    while (validarTextoZona(nombre) == 0);

    do
    {
        printf("ingrese la tematica de la zona: ");
        scanf(" %99[^\n]", tematica);

        if (validarTextoZona(tematica) == 0)
        {
            printf("tematica invalida\n");
        }
    }
    while (validarTextoZona(tematica) == 0);
}

```

```

do
{
    printf("ingrese el horario de la zona(ej:09:00-18:00) ");
    scanf(" %99[^\n]", horario);

    if (validarHorarioZona(horario) == 0)
    {
        printf("Horario invalido. use formato ej:09:00-18:00\n");
    }
}
while (validarHorarioZona(horario) == 0);

do
{
    printf("ingrese la capacidad maxima de la zona: ");
    scanf(" %d", &capacidadMax);

    if (capacidadMax <= 0)
    {
        printf("capacidad invalida, ingrese un numero mayor a 0\n");
    }
}
while (capacidadMax <= 0);

codigo = generarCodigoZona();
nuevaZona = crearZona(nombre, codigo, tematica, capacidadMax, 0,
horario);

if (nuevaZona == NULL)
{
    printf("no se pudo crear la zona\n");
    return;
}

parque->zonas[parque->cantidadZonas] = nuevaZona;
parque->cantidadZonas++;

printf("zona creada con el codigo %d\n", codigo);
}

int calcularOcupacionZona(struct Zona* zona)
{
    int contadorOcupacion = 0;
    struct NodoAtraccion* rec;
    if (zona == NULL)
    {
        printf("Error zona invalida\n");
        return 0;
    }
    /*nodo fantasma*/
    rec = zona->headAtracciones->sig;
    while (rec != NULL)
    {
        if (rec->datosAtraccion != NULL)

```

```

        {
            contadorOcupacion += rec->datosAtraccion->ocupacionActual;
        }
        rec = rec->sig;
    }
    return contadorOcupacion;
}

/*funcion que recibe una entrada y un int del nuevo estado, cambia el
estado actual de la entrada al estado recibido*/
void cambiarEstadoEntrada(struct Entrada* entrada, int nuevoEstado)
{
    entrada->estado = nuevoEstado;
}

/*funcion que recibe una entrada y un int del nuevo valor, cambia el valor
actual de la entrada al valor recibido*/
void cambiarValorEntrada(struct Entrada* entrada, int nuevoValor)
{
    entrada->valor = nuevoValor;
}

/*funcion que recibe una entrada y un int del nuevo valor, cambia el valor
actual de la entrada al valor recibido*/
void cambiarTipoEntrada(struct Entrada* entrada, int nuevoTipo)
{
    entrada->tipo = nuevoTipo;
}

/*funcion que recibe una entrada y un int de la operacion a realizar,
luego pide el input al usuario y llama otra funcion para modificar el
atributo con el input recibido*/
void modificarEntrada(struct Entrada* entrada, int operacion)
{
    int elemtmp;

    if (entrada == NULL)
    {
        return;
    }

    if (operacion == 2)
    {
        do
        {
            printf("Ingrese el valor deseado: \n");
            scanf(" %d", &elemtmp);

            if (elemtmp <= 0)
            {
                printf("valor invalido, debe ser mayor a 0\n");
            }
        }
    }
}

```

```

        while (elemtmp <= 0);
        cambiarValorEntrada(entrada, elemtmp);
    }
    else if (operacion == 3)
    {
        do
        {
            printf("ingrese el tipo deseado: \n");
            printf("1. normal\n");
            printf("2. pase Rapido\n");
            printf("3. familiar\n");
            printf("4. infantil\n");
            scanf(" %d", &elemtmp);

            if (elemtmp < 1 || elemtmp > 4)
            {
                printf("tipo invalido, ingrese una opcion entre 1 y 4
porfavor\n");
            }
        }
        while (elemtmp < 1 || elemtmp > 4);
        cambiarTipoEntrada(entrada, elemtmp);
    }
    else if (operacion == 4)
    {
        do
        {
            printf("ingrese el estado deseado: \n");
            printf("1. activa\n");
            printf("2. utilizada\n");
            printf("3. anulada\n");
            printf("4. vencida\n");
            scanf(" %d", &elemtmp);

            if (elemtmp < 1 || elemtmp > 4)
            {
                printf("estado invalido, ingrese una opcion entre 1 y
4\n");
            }
        }
        while (elemtmp < 1 || elemtmp > 4);
        cambiarEstadoEntrada(entrada, elemtmp);
    }
}

/*funcion para agregar un usuario nuevo a la lista de usuarios de una
entrada*/
void agregarUsuarioEntrada(struct Entrada* entrada, struct Usuario*
usuario)
{
    struct NodoUsuarioEntrada* nuevoNodo;
    struct NodoUsuarioEntrada* rec;
    if (entrada == NULL || usuario == NULL) { return; }

```

```

        nuevoNodo = (struct NodoUsuarioEntrada*)malloc(sizeof(struct
NodoUsuarioEntrada));
        if (nuevoNodo == NULL) { return; }

        nuevoNodo->datosUsuario = usuario;
        nuevoNodo->sig = NULL;

        rec = entrada->headUsuarios;

        while (rec->sig != NULL)
        {
            rec = rec->sig;
        }

        rec->sig = nuevoNodo;
    }

```

```

void registrarSalidaUsuarioParque(struct Usuario* usuario)
{
    /*validar si el usuario existe*/
    if (usuario == NULL)
    {
        printf("El usuario no existe!\n");
        return;
    }
    /*validar si el usuario ya esta fuera del parque*/
    if (usuario->estaEnParque == 0)
    {
        printf("El usuario ya esta fuera del parque\n");
        return;
    }
    /*sacar al usuario del parque*/
    usuario->estaEnParque = 0;
    usuario->estuvoEnParque = 1;
}

```

/*esta funcion se tiene que llamar en un loop si la entrada es de tipo familiar, porque solo actua sobre un usuario*/

```

void registrarIngresoUsuarioParque(struct Parque* parque, struct Usuario*
usuario)
{
    /*validar si el usuario existe*/
    if (usuario == NULL)
    {
        printf("El usuario no existe!\n");
        return;
    }
    /*validar si el usuario ya esta dentro del parque*/
    if (usuario->estaEnParque == 1)
    {
        printf("El usuario ya esta dentro del parque\n");
        return;
    }
    /*meter el usuario al parque*/
}

```

```

    usuario->estaEnParque = 1;
    /*registrar al usuario como visita de hoy*/
    parque->visitantesHoy++;
}

void usarEntrada(struct Entrada* entrada, struct Usuario* usuario)
{
    /*verificar si la entrada existe*/
    if (entrada == NULL)
    {
        printf("La entrada no existe!\n");
        return;
    }

    /*verificar si la entrada ya esta activa*/
    if (entrada->estado == 1)
    {
        printf("La entrada ya esta activada!\n");
        return;
    }

    /*activar entrada*/
    entrada->estado = 1;
}

int contarEntradasUsadas(struct NodoEntrada* raizEntrada)
{
    if (raizEntrada == NULL)
    {
        return 0;
    }
    if (raizEntrada->datosEntrada != NULL)
    {
        if (raizEntrada->datosEntrada->estado == 2 ||
raizEntrada->datosEntrada->estado == 3)
        {
            return 1 + contarEntradasUsadas(raizEntrada->izq) +
contarEntradasUsadas(raizEntrada->der);
        }
        else
        {
            return 0 + contarEntradasUsadas(raizEntrada->izq) +
contarEntradasUsadas(raizEntrada->der);
        }
    }
    else
    {
        return 0 + contarEntradasUsadas(raizEntrada->izq) +
contarEntradasUsadas(raizEntrada->der);
    }
}

int calcularIngresosTotales(struct NodoEntrada* raizEntrada)
{

```



```

int ingresosIzq, ingresosDer, NodoIngresos;
if (raizEntrada == NULL)
{
    return 0;
}

ingresosIzq = calcularIngresosTotales(raizEntrada->izq);
if (raizEntrada != NULL)
{
    NodoIngresos = raizEntrada->datosEntrada->valor;
}
ingresosDer = calcularIngresosTotales(raizEntrada->der);

return ingresosIzq + NodoIngresos + ingresosDer;
}

int contarEntradasVendidas(struct NodoEntrada* raizEntrada)
{
    if (raizEntrada == NULL)
    {
        return 0;
    }
    if (raizEntrada->datosEntrada != NULL)
    {
        return 1 + contarEntradasVendidas(raizEntrada->izq) +
        contarEntradasVendidas(raizEntrada->der);
    }
    else
    {
        return 0 + contarEntradasVendidas(raizEntrada->izq) +
        contarEntradasVendidas(raizEntrada->der);
    }
}

int contarVisitantesTotales(struct NodoUsuario* headUsuarios)
{
    struct NodoUsuario* rec;
    int contador = 0;
    if (headUsuarios == NULL)
    {
        printf("Usuarios invalidos\n");
        return 0;
    }
    /*Nodo Fantasma*/
    rec = headUsuarios->sig;
    while (rec != NULL)
    {
        if (rec->datosUsuario->estaEnParque != 0 ||
rec->datosUsuario->estuvoEnParque != 0)
        {
            contador++;
        }
        rec = rec->sig;
    }
}

```

```

        return contador;
    }

int contarVisitantesEnParque(struct NodoUsuario* headUsuarios)
{
    struct NodoUsuario* rec;
    int contadorVisitantes = 0;
    if (headUsuarios == NULL)
    {
        printf("Usuarios invalidos\n");
        return contadorVisitantes;
    }
    /*Nodo fantasma*/
    rec = headUsuarios->sig;
    while (rec != NULL)
    {
        if (rec->datosUsuario != NULL)
        {
            if (rec->datosUsuario->estaEnParque != 0)
            {
                contadorVisitantes++;
            }
        }
        rec = rec->sig;
    }
    return contadorVisitantes;
}

struct Atraccion** listarAtraccionesMalas(struct NodoAtraccion*
headAtracciones, int* contadorMalas)
{
    struct Atraccion** atraccionesMalas = NULL;
    struct NodoAtraccion* rec;

    if (contadorMalas == NULL)
    {
        printf("Error contador invalido\n");
        return atraccionesMalas;
    }
    *contadorMalas = 0;

    if (headAtracciones == NULL)
    {
        printf("Error Atracciones invalidas\n");
        return atraccionesMalas;
    }
    /*Nodo Fantasma*/
    rec = headAtracciones->sig;
    while (rec != NULL)
    {
        if (rec->datosAtraccion != NULL)
        {
            if (rec->datosAtraccion->estado != 1)
            {

```

```

        /*Agrandamos la memoria del arreglo , indexamos el dato en
el arreglo */
        atraccionesMalas = (struct
Atraccion**)realloc(atraccionesMalas,
        ((*contadorMalas) + 1) * sizeof(struct Atraccion*));
        atraccionesMalas[(*contadorMalas)] = rec->datosAtraccion;
        (*contadorMalas)++;
    }
    }
    rec = rec->sig;
}
return atraccionesMalas;
}

```

```

/* INICIO DE ZONA DE FUNCIONES CON PRINTS Q CONECTAN EL MENU */
void menuModificarZonas(struct Parque* parque)
{
    int codigo = 0;
    int opcion = 0;
    int capacidadNueva = 0;
    char nombreNuevo[50];
    char tematicaNueva[50];
    char horarioNuevo[50];
    struct Zona *zonaModificar;

    printf("zonas disponibles: \n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona que desea modificar:\n");
    scanf("%d", &codigo);

    zonaModificar = buscarZonaPorCodigo(parque,codigo);

    if (zonaModificar == NULL)
    {
        printf("esa zona no existe\n");
        return;
    }

    while (opcion != 5)
    {
        printf("MENU DE MODIFICAR ZONA\n");
        printf("1. cambiar nombre\n");
        printf("2. cambiar tematica\n");
        printf("3. cambiar horario\n");
        printf("4. cambiar capacidad maxima\n");
        printf("5. salir\n");
        printf("seleccione una opcion valida:\n");
        scanf("%d", &opcion);

        if (opcion == 1)

```

```

{
    do
    {
        printf("ingrese el nuevo nombre: \n");
        scanf("%s", nombreNuevo);
    }while (validarTextoZona(nombreNuevo) == 0);

    if (cambiarNombreZona(zonaModificar,nombreNuevo) == 1)
    {
        printf("el nombre ha sido modificado!\n");
    } else
    {
        printf("error al modificar el nombre\n");
    }
}

if (opcion == 2)
{
    do
    {
        printf("ingrese la nueva tematica de la zona: \n");
        scanf("%s", tematicaNueva);
    }while (validarTextoZona(tematicaNueva) == 0);

    if (cambiarTematicaZona(zonaModificar, tematicaNueva) == 1)
    {
        printf("la tematica ha sido modificada!\n");
    } else
    {
        printf("error al modificar la tematica\n");
    }
}

if (opcion == 3)
{
    do
    {
        printf("ingrese el nuevo horario respetando el formato,
ej: 09:00-18:00\n");
        scanf("%s", horarioNuevo);
    }while (validarHorarioZona(horarioNuevo) == 0);

    if (cambiarHorarioZona(zonaModificar,horarioNuevo)== 1)
    {
        printf("el horario ha sido modificado! \n");
    } else
    {
        printf("error al modificar horario\n");
    }
}

if (opcion == 4)
{
    while (capacidadNueva <= 0)

```

```

        {
            printf("ingrese la nueva capacidad maxima: \n");
            scanf("%d", &capacidadNueva);
        }

        if (cambiarCapacidadZona(zonaModificar, capacidadNueva) == 1)
        {
            printf("la capacidad maxima ha sido modificada! \n");
        } else
        {
            printf("error al modificar la capacidad maxima\n");
        }
    }

    if (opcion < 1 || opcion > 5)
    {
        printf("opcion invalida\n");
    }
}

void menuBuscarZona(struct Parque* parque)
{
    struct Zona* zonaMostrar;
    int codigo;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }
    printf("Ingrese el codigo de la zona que desea buscar : \n");
    scanf("%d", &codigo);

    zonaMostrar = buscarZonaPorCodigo(parque, codigo);
    if (zonaMostrar == NULL)
    {
        printf("Error al encontrar Zona \n");
        return;
    }
    mostrarZona(zonaMostrar, parque);
    return;
}

void menuEliminarZona(struct Parque* parque)
{
    struct Zona* zonaEliminar;
    int codigo;
    if (parque == NULL)
    {
        printf("Parque no valido\n");
        return;
    }
    printf("Ingrese el Codigo de la zona que desea eliminar : \n");
    scanf("%d", &codigo);

```

```

zonaEliminar = buscarZonaPorCodigo(parque, codigo);
if (zonaEliminar == NULL)
{
    printf("Error al intentar eliminar zona vuelva a intentarlo \n");
    return;
}
if (eliminarZona(parque, zonaEliminar) == 1)
{
    printf("Zona eliminada con exito\n");
    return;
}
else
{
    printf("Error al intentar eliminar zona vuelva a intentarlo \n");
    return;
}
}

void menuBuscarUsuarioPorId(struct Parque* parque)
{
    int idBuscar;
    struct Usuario* usuarioBuscado;

    printf("ingrese el ID del usuario a buscar: ");
    scanf(" %d", &idBuscar);

    usuarioBuscado = buscarUsuarioPorId(parque, idBuscar);

    if (usuarioBuscado == NULL)
    {
        printf("usuario no existe\n");
    }
    else
    {
        printf("usuario encontrado:\n");
        listarUsuario(usuarioBuscado);
        printf("\n");
    }
}

void menuEliminarUsuario(struct Parque* parque)
{
    int idEliminar;
    int resultado;

    printf("ingrese el ID del usuario a eliminar: ");
    scanf(" %d", &idEliminar);

    resultado = eliminarUsuario(parque, idEliminar);

    if (resultado == 1)
    {
        printf("usuario eliminado\n");
    }
}

```

```

    }
    else
    {
        printf("usuario no existe \n");
    }
}

void menuModificarUsuario(struct Parque* parque)
{
    struct Usuario* usuarioModificar;
    char nombre[100];
    int edad;
    float estatura;
    int id;
    int resultado;

    printf("\nBienvenido al menu de MODIFICACION de usuario!\n");

    printf("Ingrese el ID del usuario que desea modificar: ");
    scanf(" %d", &id);
    usuarioModificar = buscarUsuarioPorId(parque, id);

    if (usuarioModificar == NULL)
    {
        printf("Usuario con id: %d no encontrado.\n", id);
        return;
    }

    printf("Usted esta modificando al usuario de nombre: %s\n",
usuarioModificar->nombre);

    do
    {
        printf("Ingrese el nuevo nombre del usuario (Nombre Apellido): ");
        scanf(" %99[^\n]", nombre);

        if (validarNombreCompleto(nombre) == 0)
        {
            printf(
                "Nombre invalido, debe ingresar un nombre separado por
espacio, nombre y apellido con 4 letras minimo cada uno.\n");
        }
    }
    while (validarNombreCompleto(nombre) == 0);

    do
    {
        printf("Ingrese la nueva edad del usuario: ");
        scanf(" %d", &edad);

        if (validarEdad(edad) == 0)
        {
            printf("Edad invalida. Intente nuevamente.\n");
        }
    }

```

```

    }
    while (validarEdad(edad) == 0);

do
{
    printf("Ingrese la nueva estatura del usuario ej:(1.78) ");
    scanf(" %f", &estatura);

    if (validarEstatura(estatura) == 0)
    {
        printf("Estatura no valida. debe estar entre 0.5m y 2.5m\n");
    }
}
while (validarEstatura(estatura) == 0);

resultado = modificarUsuario(parque, id, nombre, edad, estatura);
if (resultado == 1)
{
    printf("Usuario modificado con exito!");
    return;
}
printf("Usuario no ha podido ser modificado.");
}

void menuAgregarEntrada(struct Parque* parque)
{
    int valortmp;
    int tipotmp;
    int estadotmp;
    struct Entrada* entradaNueva;
do
{
    printf("ingresar valor de entrada: ");
    scanf("%d", &valortmp);
    if (valortmp <= 0)
    {
        printf("valor invalido, debe ser mayor a 0\n");
    }
}
while (valortmp <= 0);

do
{
    printf("ingresar tipo de entrada: \n");
    printf("1. Normal\n");
    printf("2. Pase Rapido\n");
    printf("3. Familiar\n");
    printf("4. Infantil\n");
    scanf("%d", &tipotmp);

    if (tipotmp < 1 || tipotmp > 4)
    {

```



```

        printf("tipo invalido, ingrese una opcion de las mostradas en
pantalla\n");
    }
}
while (tipotmp < 1 || tipotmp > 4);

estadotmp = 1;

entradaNueva = crearEntrada(valortmp, tipotmp, estadotmp);

if (entradaNueva == NULL)
{
    printf("error al crear entrada\n");
    return;
}

    parque->raizEntradas->izq = agregarEntrada(parque->raizEntradas->izq,
entradaNueva);
    printf("entrada agregada con id: %d\n", entradaNueva->id);
    printf("Estado inicial: Activa\n");
}

struct Entrada* menuBuscarEntradaPorId(struct Parque* parque)
{
    int idtmp;
    struct Entrada* buscadotmp = NULL;
    printf("Ingresa id de la entrada: ");
    scanf(" %d", &idtmp);
    buscadotmp = buscarEntradaPorId(parque->raizEntradas->izq, idtmp);
    if (buscadotmp != NULL)
    {
        listarEntrada(buscadotmp);
        return buscadotmp;
    }
    else
    {
        printf("Entrada con id: %d no existe!\n", idtmp);
        return NULL;
    }
}

void menuListarEntradas(struct Parque* parque)
{
    int selecttmp;
    printf("1. Listar todas las entradas del sistema\n");
    printf("2. Listar entrada individual\n");
    printf("Porfavor digite una opcion valida del menu: ");
    scanf(" %d", &selecttmp);
    if (selecttmp == 1)
    {
        listarTodasLasEntradas(parque->raizEntradas->izq);
    }
    else

```

```

    {
        menuBuscarEntradaPorId(parque);
    }
}

void menuCambiarEstadoEntrada(struct Parque* parque)
{
    struct Entrada* buscadotmp = menuBuscarEntradaPorId(parque);
    if (buscadotmp == NULL)
    {
        printf("Entrada no encontrada \n");
    }
    else
    {
        int selecttmp;
        int estadoActual = buscadotmp->estado;

        printf("Estado actual de la entrada: %s\n",
obtenerEstadoEntrada(estadoActual));

        printf("1. Activa\n2. Utilizada\n3. Anulada\n4. Vencida\n");
        printf("Porfavor digite una opcion valida de estado: ");
        scanf(" %d", &selecttmp);
        if (selecttmp < 1 || selecttmp > 4)
        {
            printf("Error, opcion invalida! \n");
            return;
        }
        cambiarEstadoEntrada(buscadotmp, selecttmp);
        printf("Estado nuevo de la entrada: %s\n",
obtenerEstadoEntrada(buscadotmp->estado));
    }
}

void menuModificarEntrada(struct Parque* parque)
{
    listarTodasLasEntradas(parque);
    struct Entrada* buscadotmp = menuBuscarEntradaPorId(parque);
    if (buscadotmp == NULL)
    {
        printf("Entrada no encontrada \n");
    }
    else
    {
        int selecttmp;
        printf("Modificar la entrada: \n1. Valor\n2. Tipo\n3. Estado\n");
        printf("Porfavor digite una opcion valida para modificar: ");
        scanf(" %d", &selecttmp);
        if (selecttmp < 1 || selecttmp > 3)
        {
            printf("Error, opcion invalida! \n");
            return;
        }
        modificarEntrada(buscadotmp, selecttmp + 1);
    }
}

```

```

    }
}

void flujoUsuario(struct Parque* parque, int operacion)
{
    struct Entrada* buscadotmp;
    struct NodoUsuarioEntrada *rec;

    if (parque->raizEntradas->izq == NULL)
    {
        printf("no hay entradas registradas\n");
        return;
    }

    printf("Entradas disponibles:\n");
    listarTodasLasEntradas (parque->raizEntradas->izq);

    buscadotmp = menuBuscarEntradaPorId(parque);

    if (buscadotmp == NULL)
    {
        printf("Entrada no encontrada \n");
        return;
    }

    rec = buscadotmp->headUsuarios->sig;

    if (operacion == 1)
    {
        if (buscadotmp->estado != 1)
        {
            printf("esta entrada no esta activa, estado actual: %s\n",
obtenerEstadoEntrada (buscadotmp->estado));
            return;
        }
    }

    if (rec == NULL)
    {
        printf("esta entrada no tiene usuarios asignados\n");
        return;
    }

    while (rec != NULL)
    {
        if (operacion == 1)
        {
            registrarIngresoUsuarioParque (parque, rec->datosUsuario);
        }
        if (operacion == 2)
        {
            registrarSalidaUsuarioParque (rec->datosUsuario);
        }
    }
}

```

```

        rec = rec->sig;
    }

    if (operacion == 1)
    {
        buscadotmp->estado = 2;
        printf("entradas validas, usuarios ingresados al parque\n");
    }

    if (operacion == 2)
    {
        buscadotmp->estado = 3;
        printf("entradas validas, usuarios salieron del parque\n");
    }
}

void contarAtraccionesNoOperativasPorZona(struct Parque *parque) {
    int i;
    printf("\n--- Atracciones no operativas por zona ---\n");
    for (i = 0; i < parque->cantidadZonas; i++)
    {
        int cantMalas = 0;
        struct Atraccion** malas =
listarAtraccionesMalas(parque->zonas[i]->headAtracciones, &cantMalas);
        if (cantMalas > 0)
        {
            printf("Zona %s: %d atraccion(es) no operativa(s)\n",
                parque->zonas[i]->nombre, cantMalas);
        }
    }
}

void menuCerrarParque(struct Parque *parque) {
    struct NodoAtraccion *rec;

    printf("\n===== \n");
    printf("      RESUMEN DEL DIA - IBCLandia\n");
    printf("===== \n");

    printf("Visitantes que ingresaron hoy:  %d\n", parque->visitantesHoy);
    printf("Visitantes totales del dia:      %d\n",
contarVisitantesTotales(parque->headUsuarios));
    printf("Entradas vendidas:                  %d\n",
contarEntradasVendidas(parque->raizEntradas->izq));
    printf("Entradas usadas:                   %d\n",
contarEntradasUsadas(parque->raizEntradas->izq));
    printf("Ingresos totales del dia:          $%d\n",
calcularIngresosTotales(parque->raizEntradas->izq));

    contarAtraccionesNoOperativasPorZona(parque);
    parque->visitantesHoy = 0;
    printf("\nParque cerrado. Hasta manana!\n");
    printf("===== \n");
}

```

```

void menuFlujoUsuarios(struct Parque* parque)
{
    int selecttmp;
    printf("Operaciones disponibles:\n");
    printf("1. Ingresar usuarios al parque\n");
    printf("2. Sacar usuarios del parque\n");
    printf("Digite una opcion valida: ");
    scanf(" %d", &selecttmp);
    flujoUsuario(parque, selecttmp);
}

void menuAgregarUsuarioAEntrada(struct Parque* parque)
{
    struct NodoUsuarioEntrada *rec;
    printf("entradas disponibles: \n");
    listarTodasLasEntradas(parque->raizEntradas->izq);

    struct Entrada* buscadotmp = menuBuscarEntradaPorId(parque);
    if (buscadotmp == NULL)
    {
        printf("Entrada no encontrada \n");
    }
    else
    {
        rec = buscadotmp->headUsuarios->sig;
        int selecttmp;
        printf("Agregar usuario existente o nuevo:\n");
        printf("1. existente\n");
        printf("2. nuevo\n");
        printf("opcion: ");
        scanf("%d", &selecttmp);
        if (selecttmp < 1 || selecttmp > 2)
        {
            printf("Error, opcion invalida! \n");
            return;
        }
        else if (selecttmp == 1)
        {
            int idBuscar;

            listarTodosLosUsuarioSimple(parque);

            printf("Ingrese el id del usuario: ");
            scanf("%d", &idBuscar);

            struct Usuario* usuariotmp = buscarUsuarioPorId(parque,
idBuscar);

            if (usuariotmp == NULL)
            {
                printf("usuario no encontrado\n");
                return;
            }
        }
    }
}

```

```

    }

    while (rec != NULL)
    {
        if (usuariotmp->id == rec->datosUsuario->id)
        {
            printf("el usuario ya se encuentra en la fila!\n");
            return;
        }
    }

    agregarUsuarioEntrada(buscadotmp, usuariotmp);
    printf("usuario agregado correctamente a la entrada %d\n",
buscadotmp->id);
}
else if (selecttmp == 2)
{
    struct Usuario* usuariotmp = leerDatosCrearUsuario();
    agregarUsuario(parque, usuariotmp);
    agregarUsuarioEntrada(buscadotmp, usuariotmp);
}
}

void menuEliminarEntrada(struct Parque* parque)
{
    struct Entrada* buscadotmp = menuBuscarEntradaPorId(parque);
    if (buscadotmp == NULL)
    {
        printf("Entrada no encontrada \n");
    }
    else
    {
        eliminarEntrada(&parque->raizEntradas->izq, buscadotmp->id);
        printf("Entrada eliminada \n");
    }
}

void menuAgregarAtraccion(struct Parque* parque)
{
    int codigoZona;
    struct Zona* zonaBuscada;

    listarZonasSimple(parque);

    printf("Ingrese el codigo de la zona donde agregar la atraccion: ");
    scanf(" %d", &codigoZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

    if (zonaBuscada == NULL)
    {
        printf("Zona no encontrada \n");
    }
}

```

```

        return;
    }

    agregarAtraccion(zonaBuscada->headAtracciones);
}

void menuEliminarAtraccion(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    int resultado;
    struct Zona* zonaBuscada;

    printf("Zonas disponibles:\n");
    listarZonas(parque);

    printf("ingrese codigo de la zona: ");
    scanf(" %d", &codigoZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

    if (zonaBuscada == NULL)
    {
        printf("codigo invalido\n");
        return;
    }

    printf("Atracciones disponibles en la zona %d:\n",
zonaBuscada->codigo);
    listarAtraccionesSimple(zonaBuscada->headAtracciones);

    printf("ingrese el codigo de la atraccion que desea eliminar: ");
    scanf(" %d", &codigoAtraccion);

    resultado = eliminarAtraccion(zonaBuscada->headAtracciones,
codigoAtraccion);

    if (resultado == 1)
    {
        printf("atraccion eliminada!\n");
    }
    else
    {
        printf("la atraccion no existe.\n");
    }
}

void menuListarAtracciones(struct Parque* parque)
{
    int zonaCode;
    struct Zona* zonaBuscada;

    printf("zonas disponibles: \n");
    listarZonasSimple(parque);
}

```

```

        printf("ingrese el codigo de la zona que desea listar sus atracciones:
");
        scanf(" %d", &zonaCode);

        zonaBuscada = buscarZonaPorCodigo(parque, zonaCode);

        if (zonaBuscada == NULL)
        {
            printf("no se encontro la zona.\n");
            return;
        }

        listarAtracciones(zonaBuscada->headAtracciones);
    }

void menuBuscarAtraccion(struct Parque* parque)
{
    int codeZona;
    int codeAtraccion;
    struct Zona* zonaBuscar;
    struct Atraccion* atraccionBuscar;

    printf("zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codeZona);

    zonaBuscar = buscarZonaPorCodigo(parque, codeZona);

    if (zonaBuscar == NULL)
    {
        printf("zona no encontrada\n");
        return;
    }

    printf("ingrese el codigo de la atraccion que desea buscar: ");
    scanf(" %d", &codeAtraccion);

    atraccionBuscar =
    buscarAtraccionPorCodigo(zonaBuscar->headAtracciones, codeAtraccion);

    if (atraccionBuscar == NULL) return;

    printf("atraccion encontrada!\n");
    printf("Nombre: %s\n", atraccionBuscar->nombre);
    printf("Codigo: %d\n", atraccionBuscar->codigo);
    printf("capacidadMax: %d\n", atraccionBuscar->capacidadMax);
    printf("ocupacionActual: : %d\n", atraccionBuscar->ocupacionActual);
    printf("estado: %d\n", atraccionBuscar->estado);
}

```



```

void menuModificarAtraccion(struct Parque* parque)
{
    int codeZona;
    int codeAtraccion;
    int opcion;
    int ocupacionAntes;

    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codeZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codeZona);

    if (zonaBuscada == NULL)
    {
        printf("zona no existe\n");
        return;
    }

    printf("ingrese el codigo de la atraccion a modificar: ");
    scanf(" %d", &codeAtraccion);

    atraccionBuscada =
    buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codeAtraccion);

    if (atraccionBuscada == NULL)
    {
        printf("atraccion no existe\n");
        return;
    }

    do
    {
        printf("modificando atraccion de codigo %d:\n",
atraccionBuscada->codigo);
        printf("1. nombre\n");
        printf("2. capacidad maxima\n");
        printf("3. estado\n");
        printf("4. ya no quiero seguir modificando\n");
        printf("ingrese una opciones de las listadas: ");
        scanf(" %d", &opcion);

        if (opcion >= 1 && opcion <= 3)
        {
            ocupacionAntes = atraccionBuscada->ocupacionActual;
            modificarAtraccion(atraccionBuscada, opcion);

            if (opcion == 3 && atraccionBuscada->estado != 1)
            {
                zonaBuscada->ocupacionActual =
zonaBuscada->ocupacionActual - ocupacionAntes;

```

```

        if (zonaBuscada->ocupacionActual < 0)
        {
            zonaBuscada->ocupacionActual = 0;
            /*parche temporal porque aveces queda negativo la
resta por alguna razon */
        }
    }
    printf("atraccion modificada!\n");
}
else
{
    if (opcion != 4)
    {
        printf("tu opcion es invalida\n");
    }
}
}
while (opcion != 4);

printf("volviendo al menu de atracciones!\n");
}

void menuAgregarUsuarioAFila(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    int idUsuario;
    int resultado;
    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;
    struct Usuario* usuarioBuscado;

    printf("Zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codigoZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

    if (zonaBuscada == NULL)
    {
        printf("la zona de codigo %d, no existe.\n", codigoZona);
        return;
    }

    printf("atracciones disponibles en la zona %d: \n", codigoZona);
    listarAtraccionesSimple(zonaBuscada->headAtracciones);

    printf("ingrese el codigo de la atraccion a la que desea ingresar el
usuario: ");
    scanf(" %d", &codigoAtraccion);

```

```

                                atraccionBuscada =
buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codigoAtraccion);

    if (atraccionBuscada == NULL)
    {
        printf("la atraccion de codigo %d no existe\n", codigoAtraccion);
        return;
    }

    printf("lista de usuarios:\n");
    listarTodosLosUsuarioSimple(parque);

    printf("ingrese el id del usuario que desea ingresar a una fila: ");
    scanf(" %d", &idUsuario);

    usuarioBuscado = buscarUsuarioPorId(parque, idUsuario);

    if (usuarioBuscado == NULL)
    {
        printf("usuario no ha sido encontrado\n");
        return;
    }

    if (validarUsuarioSiPuedeFila(parque, atraccionBuscada, idUsuario) ==
0) return;

    resultado = agregarUsuarioAFila(atraccionBuscada, usuarioBuscado);
    if (resultado == 1)
    {
        printf("usuario agregado correctamente a la fila de la atraccion:
%d\n", codigoAtraccion);
    }
    if (resultado == 2)
    {
        printf("la atraccion no esta operativa.\n");
    }
    if (resultado == 0)
    {
        printf("error, usuario no ha podido ser agregado a la fila\n");
    }
}

void menuListarFilaAtraccion(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;

    printf("Zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codigoZona);

```

```

zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

if (zonaBuscada == NULL)
{
    printf("la zona de codigo %d, no existe.\n", codigoZona);
    return;
}

printf("atracciones disponibles en la zona %d: \n", codigoZona);
listarAtraccionesSimple(zonaBuscada->headAtracciones);

printf("ingrese el codigo de la atraccion: ");
scanf(" %d", &codigoAtraccion);

                                atraccionBuscada
buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codigoAtraccion); =

if (atraccionBuscada == NULL)
{
    printf("la atraccion de codigo %d no existe\n", codigoAtraccion);
    return;
}

listarFilaAtraccion(atraccionBuscada);
}

void menuBuscarUsuarioEnFila(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    int idUsuario;
    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;
    struct Usuario* usuarioBuscado;

    printf("Zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codigoZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

    if (zonaBuscada == NULL)
    {
        printf("la zona de codigo %d, no existe.\n", codigoZona);
        return;
    }

    printf("atracciones disponibles en la zona %d: \n", codigoZona);
    listarAtraccionesSimple(zonaBuscada->headAtracciones);

    printf("ingrese el codigo de la atraccion: ");

```

```

scanf(" %d", &codigoAtraccion);

                                atraccionBuscada
buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codigoAtraccion); =

if (atraccionBuscada == NULL)
{
    printf("la atraccion de codigo %d no existe\n", codigoAtraccion);
    return;
}

printf("lista de usuarios:\n");
listarTodosLosUsuarioSimple(parque);

printf("ingrese el id del usuario que desea ingresar a una fila: ");
scanf(" %d", &idUsuario);

usuarioBuscado = buscarUsuarioEnFila(atraccionBuscada, idUsuario);

if (usuarioBuscado == NULL)
{
    printf("usuario no ha sido encontrado en la fila\n");
    return;
}

printf("usuario encontrado en la fila!, sus datos son: \n");
printf("nombre: %s\n", usuarioBuscado->nombre);
printf("id: %d\n", usuarioBuscado->id);
printf("edad: %d\n", usuarioBuscado->edad);
printf("estatura: %.2f\n", usuarioBuscado->estatura);
}

void menuIniciarVueltaAtraccion(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    int resultado;
    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;

    printf("Zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");
    scanf(" %d", &codigoZona);

    zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

    if (zonaBuscada == NULL)
    {
        printf("la zona de codigo %d, no existe.\n", codigoZona);
        return;
    }
}

```

```

printf("atracciones disponibles en la zona %d: \n", codigoZona);
listarAtraccionesSimple(zonaBuscada->headAtracciones);

printf("ingrese el codigo de la atraccion: ");
scanf(" %d", &codigoAtraccion);

                                atraccionBuscada      =
buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codigoAtraccion);

if (atraccionBuscada == NULL)
{
    printf("la atraccion de codigo %d no existe\n", codigoAtraccion);
    return;
}

resultado = iniciarVueltaAtraccion(atraccionBuscada);

if (resultado > 0)
{
    zonaBuscada->ocupacionActual = zonaBuscada->ocupacionActual +
resultado;
    printf("vuelta iniciada correctamente\n");
    printf("personas que entraron a la atraccion: %d\n", resultado);
}
else if (resultado == 0)
{
    printf("la fila de esta atraccion esta vacia\n");
}
else if (resultado == -1)
{
    printf("la atraccion no esta operativa\n");
}
else if (resultado == -2)
{
    printf("la atraccion ya esta en una vuelta\n");
}
else
{
    printf("error desconocido\n");
}
}

void menuTerminarVueltaAtraccion(struct Parque* parque)
{
    int codigoZona;
    int codigoAtraccion;
    int resultado;
    struct Zona* zonaBuscada;
    struct Atraccion* atraccionBuscada;

    printf("Zonas disponibles:\n");
    listarZonasSimple(parque);

    printf("ingrese el codigo de la zona: ");

```

```

scanf(" %d", &codigoZona);

zonaBuscada = buscarZonaPorCodigo(parque, codigoZona);

if (zonaBuscada == NULL)
{
    printf("la zona de codigo %d, no existe.\n", codigoZona);
    return;
}

printf("atracciones disponibles en la zona %d: \n", codigoZona);
listarAtraccionesSimple(zonaBuscada->headAtracciones);

printf("ingrese el codigo de la atraccion: ");
scanf(" %d", &codigoAtraccion);

                                atraccionBuscada =
buscarAtraccionPorCodigo(zonaBuscada->headAtracciones, codigoAtraccion);

if (atraccionBuscada == NULL)
{
    printf("la atraccion de codigo %d no existe\n", codigoAtraccion);
    return;
}

resultado = terminarVueltaAtraccion(atraccionBuscada);

if (resultado > 0)
{
    zonaBuscada->ocupacionActual = zonaBuscada->ocupacionActual -
resultado;

    if (zonaBuscada->ocupacionActual < 0)
    {
        zonaBuscada->ocupacionActual = 0;
    }

    printf("vuelta terminada! espero hayan disfrutado la
atraccion!\n");
    printf("personas que salieron de la atraccion: %d\n", resultado);
}
else if (resultado == 0)
{
    printf("esta atraccion no esta en una vuelta\n");
}
}

void menuMostrarOcupacionZona(struct Parque* parque)
{
    int codigo, ocupacionZona;
    struct Zona* zona;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
    }
}

```

```

        return;
    }

    printf("Zonas disponibles: \n");
    listarZonasSimple(parque);

    printf("Ingrese el codigo de la zona que desea calcular su ocupacion :
\n");
    scanf("%d", &codigo);
    zona = buscarZonaPorCodigo(parque, codigo);
    if (zona == NULL)
    {
        printf("Error al buscar zona\n");
        return;
    }
    ocupacionZona = calcularOcupacionZona(zona);
    printf("La zona tiene una ocupacion de %d persona(s).\n",
ocupacionZona);
    return;
}

void menuCalcularIngresosTotales(struct Parque* parque)
{
    int ingresosTotales = 0;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }
    ingresosTotales = calcularIngresosTotales(parque->raizEntradas->izq);
    printf("El ingreso total de hoy del Parque es $%d \n",
ingresosTotales);
    return;
}

void menuCalcularEntradasVendidas(struct Parque* parque)
{
    int contador = 0;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }
    contador = contarEntradasVendidas(parque->raizEntradas->izq);
    printf("La cantidad de entradas vendidas en el dia son: %d\n",
contador);
    return;
}

void menuCalcularEntradasUsadas(struct Parque* parque)
{
    int entradasUsadas = 0;
    if (parque == NULL)
    {

```



```

        printf("Parque invalido\n");
        return;
    }
    entradasUsadas = contarEntradasUsadas(parque->raizEntradas->izq);
    printf("La cantidad de entradas usadas hoy son: %d\n",
entradasUsadas);
    return;
}

void menuListarAtraccionesNoOperativas(struct Parque* parque)
{
    int codigo, cantMalas = 0, i;
    struct Zona* zona;
    struct Atraccion** AtraccionesNoOperativas;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }

    printf("Zonas Disponibles: \n");
    listarZonasSimple(parque);
    printf("Ingrese el codigo de la zona desde donde desea listar sus
Atracciones no operativas : \n");
    scanf("%d", &codigo);
    zona = buscarZonaPorCodigo(parque, codigo);
    if (zona == NULL)
    {
        printf("Error al buscar zona\n");
        return;
    }

    AtraccionesNoOperativas =
listarAtraccionesMalas(zona->headAtracciones, &cantMalas);
    if (cantMalas == 0)
    {
        printf("No hay atracciones No operativas en esta zona\n");
        return;
    }
    if (AtraccionesNoOperativas == NULL)
    {
        printf("Error al intentar conseguir atracciones no operativas\n");
        return;
    }
    printf("Atracciones No Operativas: \n");
    for (i = 0; i < cantMalas; i++)
    {
        printf("Atraccion %d\n", i + 1);
        printf("Nombre: %s\n", AtraccionesNoOperativas[i]->nombre);
        printf("Codigo: %d\n", AtraccionesNoOperativas[i]->codigo);
        printf("Estado: %d\n", AtraccionesNoOperativas[i]->estado);
        printf("Ocupacion actual: %d\n",
AtraccionesNoOperativas[i]->ocupacionActual);
        printf("Capacidad maxima: %d\n",
AtraccionesNoOperativas[i]->capacidadMax);
    }
}

```

```

        printf("-----\n");
    }
}

void menuVisitantesTotales(struct Parque* parque)
{
    int visitantes = 0;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }
    visitantes = contarVisitantesTotales(parque->headUsuarios);
    printf("La cantidad de visitantes totales de hoy es : %d\n",
visitantes);
    return;
}

void menuVisitantesEnParque(struct Parque* parque)
{
    int enParque = 0;
    if (parque == NULL)
    {
        printf("Parque invalido\n");
        return;
    }
    enParque = contarVisitantesEnParque(parque->headUsuarios);
    printf("La cantidad de visitantes dentro del parque es: %d\n",
enParque);
    return;
}

/*INICIO DE MENU ESTANDAR SOLO LLAMADAS NO LOGICA MAS ALLA DEL MENU MISMO
*/

void menuUsuarios(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de usuarios!\n");
        printf("Visitantes totales de hoy: %d\n", parque->visitantesHoy);
        printf("1. Agregar usuario\n");
        printf("2. Eliminar usuario\n");
        printf("3. Buscar usuario\n");
        printf("4. Modificar usuario\n");
        printf("5. Listar usuarios\n");
        printf("6. Controlar flujo de usuarios\n");
        printf("7. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);
    }

```

```

switch (num)
{
default:
    printf("tu opcion no es valida\n");
    break;
case 1:
    agregarUsuario(parque, leerDatosCrearUsuario());
    break;
case 2:
    menuEliminarUsuario(parque);
    break;
case 3:
    menuBuscarUsuarioPorId(parque);
    break;
case 4:
    menuModificarUsuario(parque);
    break;
case 5:
    listarTodosLosUsuarios(parque);
    break;
case 6:
    menuFlujoUsuarios(parque);
    break;
case 7:
    printf("volviendo al menu principal!\n");
    break;
}
}
while (num != 7);
}

void menuEntradas(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de entradas!\n");
        printf("1. Agregar entrada\n");
        printf("2. Eliminar entrada\n");
        printf("3. Buscar entrada\n");
        printf("4. Modificar entrada\n");
        printf("5. Listar entradas\n");
        printf("6. Cambiar estado de entrada\n");
        printf("7. Agregar usuario a entrada\n");
        printf("8. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);

        switch (num)
        {
        default:
            printf("tu opcion no es valida\n");

```

```

        break;
    case 1:
        menuAgregarEntrada (parque);
        break;
    case 2:
        menuEliminarEntrada (parque);
        break;
    case 3:
        menuBuscarEntradaPorId (parque);
        break;
    case 4:
        menuModificarEntrada (parque);
        break;
    case 5:
        menuListarEntradas (parque);
        break;
    case 6:
        menuCambiarEstadoEntrada (parque);
        break;
    case 7:
        menuAgregarUsuarioAEntrada (parque);
        break;
    case 8:
        printf("volviendo al menu principal!\n");
        break;
    }
}
while (num != 8);
}

void menuZonas(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de zonas!\n");
        printf("Zonas registradas: %d\n", parque->cantidadZonas);
        printf("1. Agregar zona\n");
        printf("2. Eliminar zona\n");
        printf("3. Buscar zona\n");
        printf("4. Modificar zona\n");
        printf("5. Listar zonas\n");
        printf("6. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);

        switch (num)
        {
            default:
                printf("tu opcion no es valida\n");
                break;
            case 1:

```

```

        agregarZona (parque);
        break;
    case 2:
        menuEliminarZona (parque);
        break;
    case 3:
        menuBuscarZona (parque);
        break;
    case 4:
        menuModificarZonas (parque);
        break;
    case 5:
        listarZonas (parque);
        break;
    case 6:
        printf("volviendo al menu principal!\n");
        break;
    }
}
while (num != 6);
}

void menuAtracciones(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de atracciones!\n");
        printf("1. Agregar atraccion\n");
        printf("2. Eliminar atraccion\n");
        printf("3. Buscar atraccion\n");
        printf("4. Modificar atraccion\n");
        printf("5. Listar atracciones\n");
        printf("6. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);

        switch (num)
        {
            default:
                printf("tu opcion no es valida\n");
                break;
            case 1:
                menuAgregarAtraccion (parque);
                break;
            case 2:
                menuEliminarAtraccion (parque);
                break;
            case 3:
                menuBuscarAtraccion (parque);
                break;
            case 4:

```

```

        menuModificarAtraccion(parque);
        break;
    case 5:
        menuListarAtracciones(parque);
        break;
    case 6:
        printf("volviendo al menu principal!\n");
        break;
    }
}
while (num != 6);
}

void menuFilas(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de filas!\n");
        printf("1. Agregar usuario a fila\n");
        printf("2. Iniciar vuelta de atraccion\n");
        printf("3. Terminar vuelta de atraccion\n");
        printf("4. Buscar usuario en fila\n");
        printf("5. Listar fila de una atraccion\n");
        printf("6. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);

        switch (num)
        {
            default:
                printf("tu opcion no es valida\n");
                break;
            case 1:
                menuAgregarUsuarioAFila(parque);
                break;
            case 2:
                menuIniciarVueltaAtraccion(parque);
                break;
            case 3:
                menuTerminarVueltaAtraccion(parque);
                break;
            case 4:
                menuBuscarUsuarioEnFila(parque);
                break;
            case 5:
                menuListarFilaAtraccion(parque);
                break;
            case 6:
                printf("volviendo al menu principal!\n");

```

```

        break;
    }
}
while (num != 6);
}

void menuReportes(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu de reportes!\n");
        printf("1. Calcular ingresos totales\n");
        printf("2. Contar entradas vendidas\n");
        printf("3. Contar entradas usadas\n");
        printf("4. Visitantes totales hoy\n");
        printf("5. Visitantes dentro del parque\n");
        printf("6. Mostrar ocupacion de Zonas\n");
        printf("7. Listar atracciones no operativas\n");
        printf("8. Volver\n");

        printf("\n Porfavor digite una opcion valida del menu: ");
        scanf(" %d", &num);

        switch (num)
        {
            default:
                printf("tu opcion no es valida\n");
                break;
            case 1:
                menuCalcularIngresosTotales(parque);
                break;
            case 2:
                menuCalcularEntradasVendidas(parque);
                break;
            case 3:
                menuCalcularEntradasUsadas(parque);
                break;
            case 4:
                menuVisitantesTotales(parque);
                break;
            case 5:
                menuVisitantesEnParque(parque);
                break;
            case 6:
                menuMostrarOcupacionZona(parque);
                break;
            case 7:
                menuListarAtraccionesNoOperativas(parque);
                break;
            case 8:
                printf("volviendo al menu principal!\n");
                break;
        }
    }
}

```

```

        }
    }
    while (num != 8);
}

void menuIBCLandia(struct Parque* parque)
{
    int num;

    do
    {
        printf("\n Bienvenido al menu!\n");
        printf("1. Menu de usuarios\n");
        printf("2. Menu de entradas\n");
        printf("3. Menu de zonas\n");
        printf("4. Menu de atracciones\n");
        printf("5. Menu de filas\n");
        printf("6. Menu de reportes\n");
        printf("7. Cerrar el parque por el dia\n");
        printf("8. Salir del menu\n");
        printf("Porfavor digite una opcion valida del menu: ");

        scanf(" %d", &num);

        switch (num)
        {
            default:
                printf("opcion no valida, mejor suerte a la proxima!\n");
                break;
            case 1:
                menuUsuarios(parque);
                break;
            case 2:
                menuEntradas(parque);
                break;
            case 3:
                menuZonas(parque);
                break;
            case 4:
                menuAtracciones(parque);
                break;
            case 5:
                menuFilas(parque);
                break;
            case 6:
                menuReportes(parque);
                break;
            case 7:
                menuCerrarParque(parque);
                break;
            case 8:
                printf("Esperamos hayas disfrutado de IBCLandia!\n");
                break;
        }
    }
}

```



```

    }
    while (num != 8);
}

int main(void)
{
    struct Parque* parque;
    parque = (struct Parque*)malloc(sizeof(struct Parque));

    if (parque == NULL)
    {
        printf("No se pudo crear el parque.\n");
        return 0;
    }
    srand(time(NULL));
    inicializarParque(parque);
    menuIBCLandia(parque);

    return 0;
}

```